



武汉大学
WUHAN UNIVERSITY

F7LY-OS

OSCOMP 设计文档

队伍名称: F7LY

队伍成员: 曹子宸、郑喆宇、何佳怡

指导老师: 蔡朝晖

2026 年 6 月

目录

第一章 F7LY-OS 内核概述	1
1.1 项目目标与 2026 年演进方向	1
1.2 当前系统能力总览	2
1.3 2026 年整体分层架构	3
1.3.1 用户态层	3
1.3.2 系统调用层 / Linux ABI	3
1.3.3 内核服务层	3
1.3.4 内核基础设施层	4
1.3.5 基础层	4
1.3.6 硬件平台层	5
1.4 参考的代码	5
1.5 我们的工作	5
1.6 项目目录结构	7
1.7 2025 与 2026 能力对照	7
第二章 机器启动	9
2.1 RISC-V 启动流程	9
2.1.1 entry.S	9
2.1.2 start.cc	9
2.1.3 主线: main() 四阶段	9
2.2 LoongArch 启动流程	11
2.2.1 入口: entry.S	11
2.2.2 main() 四阶段	11
2.3 双架构支持特色	12
2.3.1 启动完成标志	12
第三章 中断管理器	13
3.1 整体设计	13
3.2 RISC-V 中断处理路径	13
3.3 LoongArch 中断处理路径	14
3.4 陷阱分发逻辑	14

第四章 内存管理	16
4.1 内核地址空间	16
4.1.1 物理内存发现与布局	16
4.1.2 内核内存分配器	17
4.1.3 双架构页表差异	19
4.2 用户地址空间	19
4.2.1 用户地址空间布局	19
4.2.2 进程内存管理器 (ProcessMemoryManager)	21
4.2.3 VMA 组织	22
4.2.4 缺页异常处理	24
第五章 进程与线程管理	26
5.1 进程控制块 (PCB)	26
5.1.1 组成	26
5.1.2 状态机	27
5.2 进程调度	28
5.2.1 上下文切换	28
5.2.2 轮转调度	28
5.2.3 优先级调度	28
5.3 进程生命周期	29
5.3.1 创建 (fork / clone / clone3)	29
5.3.2 装载 (execve)	30
5.3.3 退出 (exit / wait)	30
5.3.4 线程	31
第六章 文件系统	33
6.1 设备抽象层	33
6.2 块设备驱动与 I/O 调度	33
6.2.1 统一 virtio-blk 框架	33
6.2.2 mClock 调度器	34
6.3 VFS 与文件对象	35
6.4 VFS 核心元数据	36
6.4.1 Buffer 层	36
6.4.2 SuperBlock、Inode 与 File	36

6.5	ext4 与 FAT32	37
6.6	文件操作	37
6.7	挂载与路径解析	37
6.8	虚拟文件与特殊设备	37
第七章	进程间通信	39
7.1	信号与定时器	39
7.2	futex	39
7.2.1	wait 与 wakeup	40
7.2.2	Robust List 与 Requeue	41
7.3	管道与 FIFO	41
7.3.1	管道语义	41
7.3.2	FIFO 管理器	41
7.4	共享内存与 memfd	42
7.4.1	SysV 共享内存	42
7.4.2	memfd	44
7.5	就绪通知	44
7.5.1	epoll	44
7.5.2	eventfd	44
第八章	系统调用	46
8.1	系统调用概述	46
8.2	执行流程	48
8.3	分发表设计	48
8.4	参数与用户内存	48
8.5	实现分层	49
8.6	新增系统调用的约束	49
第九章	网络系统模块	51
9.1	网络系统架构概述	51
9.2	核心网络协议栈与 loopback 数据面	53
9.2.1	loopback 端口表	54
9.2.2	TCP loopback 方法	55
9.2.3	UDP loopback 方法	57
9.2.4	IPv6 loopback 兼容	59

9.3	ONPS 与 VirtIO 出网路径	59
9.3.1	RX/TX 队列组织	60
9.3.2	ONPS 适配层	62
9.4	BSD Socket 接口与系统调用集成	63
9.4.1	系统调用层职责	64
9.4.2	socket_file 的状态机	65
9.4.3	批量消息与选项兼容	66
9.4.4	AF_UNIX 与 socketpair	67
9.5	小结	67
第十章	总结和展望	69
10.1	已验证的功能	69
10.1.1	连续测例通过情况	69
10.1.2	用户程序运行能力	70
10.2	AI 工具使用说明	71
10.3	总结与后续工作	72
第十一章	附录	73
11.1	附录 A PCB 字段总表	73
11.2	附录 B 系统调用实现列表	75
11.2.1	B.1 进程管理	75
11.2.2	B.2 内存管理	77
11.2.3	B.3 文件系统	78
11.2.4	B.4 信号	80
11.2.5	B.5 时间与定时器	80
11.2.6	B.6 Futex	81
11.2.7	B.7 事件通知	81
11.2.8	B.8 网络	82
11.2.9	B.9 权限	82
11.2.10	B.10 系统信息与其他	83
11.2.11	B.11 扩展属性 (xattr)	83
11.2.12	B.12 文件监控	84
11.2.13	B.13 SysV 信号量	84
11.2.14	B.14 其他高级接口	84

11.3 附录 C 虚拟文件与特殊设备节点	85
-----------------------------	----

第一章 F7LY-OS 内核概述

1.1 项目目标与 2026 年演进方向

F7LY-OS 是 C++ 编写的支持 RISC-V 与 LoongArch 双架构的宏内核模块化操作系统，面向教学、比赛、Linux ABI 兼容与连续测例评测。项目从零参考 xv6 构建，持续演进至可运行动态链接的用户程序、BusyBox 交互式 Shell 及大规模 LTP 连续测例。

2026 年的主线目标是在双架构一致性、Linux ABI 正确性、长时间运行稳定性和 I/O 与线程性能四个方向上取得实质进展。相比于 2025 年的基础版本，2026 年的内核已具备：

- 完整的动态 ELF 装载链，支持 musl / glibc 双 C 运行库和 #! 脚本解释器递归；
- 真实可用的网络协议栈，支持 loopback 与外部 IPv4 数据传输，可正常访问 web 服务；
- 双架构统一 virtio-blk 框架与 mClock 风格 I/O 调度器；
- 交互式 BusyBox ash 终端，支持控制台输入、文件浏览和脚本执行；
- 双架构、双 C 运行库的一致运行环境，覆盖常见系统工具、脚本解释器、网络程序和 C 运行库程序。

内核当前为 C++23 freestanding 环境，显式禁用异常和 RTTI (`-fno-exceptions` 、 `-fno-rtti`)，以 GCC 工具链编译。系统调用严格遵循 Linux raw syscall 规范，累计实现超过 240 个系统调用。

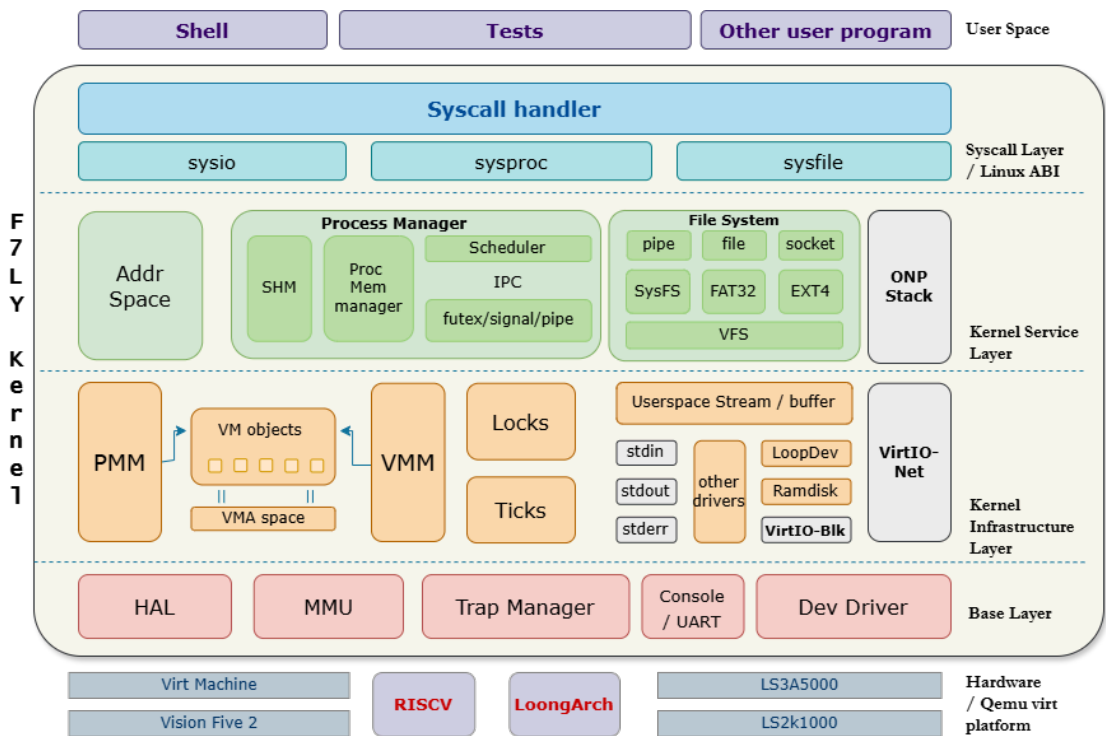


图 1-1 F7LY-OS 2026 年整体架构图

1.2 当前系统能力总览

双架构支持——RISC-V 与 LoongArch 均以 QEMU virt 平台为主要验证目标，同时在 VisionFive 2 (RISC-V) 和 LS3A5000 / LS2k1000 (LoongArch) 实机硬件上进行了启动验证。两架构共享同一套内核核心逻辑，差异封装在 HAL (kernel/ha1/) 和架构特定目录中。

内存管理——伙伴系统 (Buddy) 管理物理页，内核堆基于 liballoc 提供任意大小分配，VMA 子系统以描述符→Maple Tree 索引→后端对象 (VmObject) 三层架构管理用户地址空间。支持 mmap、munmap、mprotect、COW、惰性分配、共享内存、brk 堆和栈自动扩展。

进程与线程——统一 Pcb 抽象表示进程和线程。支持 fork、clone / clone3、execve、wait4 / waitid 完整生命周期。线程通过 CLONE_VM | CLONE_THREAD 创建，共享地址空间，支持 futex、健壮 futex、CLONE_CHILD_CLEARTID 等 pthread 所需语义。

IPC——信号 (含实时信号、sigqueue、siginfo_t)、POSIX timer、System V 共享内存 (shmget / shmat / shmdt / shmctl)、匿名管道、有名管道 (FIFO)、eventfd、memfd、epoll。

文件系统——VFS 统一文件操作接口，支持 ext4（根文件系统，基于 lwext4）和 FAT32（数据盘）。File 抽象类通过 C++ 多态支持普通文件、管道、socket、虚拟文件（/proc）和 epoll 文件。支持 ioctl、fcntl、flock 文件锁、splice/sendfile 零拷贝搬运、fanotify 通知、loop 设备、ramdisk 等。

网络——支持本机 loopback 及外部 IPv4 访问。loopback 路径由 socket_file 直接实现，不经过 VirtIO 网卡和 Ethernet/IP 层。非 loopback IPv4 流量在网络栈初始化成功后交给 ONPS，再经 virtio0 适配层和 VirtIO-Net 驱动收发完整以太网帧。BSD Socket ABI 支持 socket/socketpair、bind/listen/connect/accept、send/recv/sendmsg/sendmmsg/recvmmsg、常用 socket option、socket ioctl 兼容视图，以及 poll/epoll 就绪通知。

用户态——支持 musl 和 glibc 两种 C 运行库的动态链接程序。用户态入口包括自动启动入口和交互式 BusyBox ash，后者使用独立 ext4 rootfs 镜像，支持命令行浏览、脚本执行和正常退出。支持通过 apk 包管理器安装 Alpine Linux 软件包。

工程体系——提供双架构构建、日志保存、任务计划和能力追踪流程，使功能开发、运行验证和文档更新可复现、可回溯。

1.3 2026 年整体分层架构

F7LY-OS 的新架构图按数据流和依赖关系分为六层，自上而下为：

1.3.1 用户态层

最顶层，运行在 U-Mode。包括 BusyBox ash 交互式 Shell、自动连续测例（LTP、libcbench、iozone、lmbench、cyclictst、iperf、netperf、unixbench、lua、BusyBox、libctest）和通用用户程序。三者共享同一套 Linux ABI，但 Shell 模式使用独立 rootfs 镜像隔离交互式运行评测环境。

1.3.2 系统调用层 / Linux ABI

系统调用分三组模块：sysio（I/O 类，如 read/write/open/close/socket）、sysproc（进程类，如 fork/clone/exec/wait/signal/timer）、sysfile（文件系统类，如 mount/stat/link/xattr）。每组模块负责参数校验、权限检查和调用核心服务。底层通过统一的 syscall_handler 表分发，两架构的各系统调用号保持一致。

1.3.3 内核服务层

内核核心功能的集中承载层：

- **地址空间 (AddrSpace)**：封装进程页表、VMA 空间和 VmObject 后端对象，是内存管理的统一入口。
- **进程内存管理器 (Proc Mem manager)**：每个进程的 ProcessMemoryManager 实例，管理 ELF 段装载、堆、栈、mmap 区和共享内存附加。
- **共享内存 (SHM)**：System V 共享内存的全局管理，支持 key→段 ID 查找和跨进程附加。
- **调度器 (Scheduler)**：轮转 + 优先级调度，nice 值同时影响 CPU 和 I/O 调度。
- **IPC**：futex、信号、管道等进程间通信原语。
- **VFS**：虚拟文件系统层，统一转发到 ext4、FAT32、SysFS 等具体文件系统。pipe、file、socket 三种 File 子类在此层注册并对外暴露统一接口。

1.3.4 内核基础设施层

提供内核自身运行所需的服务和抽象：

- **物理内存管理 (PMM)**：Buddy 伙伴分配器，以 4KB 页为单位管理物理内存。
- **虚拟内存管理 (VMM)**：封装页表操作和 TLB 管理。
- **VMA 空间**：以 Maple Tree 索引加速 VMA 查找和区间搜索。
- **锁机制 (Locks)**：SpinLock 和 SleepLock，保护内核临界区。
- **时钟 (Ticks)**：timer tick 驱动调度和定时器。
- **块设备支持**：Ramdisk、LoopDev，以及通过 VirtIO-Blk 访问的磁盘。
- **标准 I/O**：stdin/stdout/stderr 经控制台/UART 与用户态交互。
- **网络基础设施**：ONPS 网络协议栈 + VirtIO-Net 驱动。

1.3.5 基础层

直接与硬件交互的底层模块：

- **HAL (硬件抽象层)**：封装 CPU 操作、上下文切换、地址转换和架构特定寄存器访问。
- **MMU**：页表硬件操作的直接封装，RISC-V 走 Sv39，LoongArch 结合 DMWIN 直映窗口。
- **Trap Manager**：统一中断/异常/系统调用入口，分发到对应处理函数。
- **Console / UART**：控制台行规程和 SBI/UART 输入输出。
- **Dev Driver**：设备驱动框架，管理字符设备、块设备和流设备的注册和访问。

1.3.6 硬件平台层

QEMU virt 机器是主要验证平台（RISC-V 和 LoongArch），同时内核已在 VisionFive 2（RISC-V 实机）、LS3A5000 和 LS2k1000（LoongArch 实机）上成功启动并进入用户态。

1.4 参考的代码

F7LY-OS 在开发过程中参考和移植了以下开源项目与第三方库：

- **xv6-2021**: MIT S081 课程的教学内核 [xv6-2021](#)。异常处理机制、内核地址布局 and 伙伴分配器算法均参考其设计，并在双架构和 C++ 环境下重新实现。
- **lwext4**: C 语言编写的 ext4 文件系统库 [lwext4](#)，经 C++ 适配与重构后作为 F7LY 的主要文件系统后端。
- **Open-NPStack (ONPS)**: 嵌入式网络协议栈 [ONPS](#)，提供 TCP/IP/UDP/ICMP 等协议的处理框架，结合 VirtIO-Net 驱动形成 F7LY 的网络子系统。
- **EASTL**: Electronic Arts 标准模板库 [EASTL](#)，在内核不支持标准 C++ 库的环境下提供 vector、list、map、hash_map 等高效数据结构。
- **liballoc**: 轻量级堆分配器，经集成后作为内核堆内存管理的细粒度分配后端。
- **XN6**: 2024 年武汉大学团队的 LoongArch xv6 移植项目 [XN6](#)，在 LoongArch 页表管理和硬件抽象层设计上提供了重要参考。
- **Starry-OS**: 2024 年 OSCOMP 冠军内核 [StarryOS](#)，在 I/O 调度和性能优化思路提供了横向对比参考。

此外，在性能优化（iozone、libcbench）阶段参考了 OSCOMP 2026 排行榜上其他参赛内核的优化思路，以及官方 autotest-for-oskernel [评测仓库](#) 中的 baseline 评测机制。

1.5 我们的工作

2026 年度工作由三个队员共同开发完成，在 2025 年基线之上进行了大量重构和新增功能。截至 2026 年 6 月，第二阶段集中开发共 96 个提交，改动 388 个文件（内核 146 个文件），系统调用绑定由 224 个增至 243 个。主要工作包括：

- **VMA 与进程地址空间重构**——引入 `VmArea` 描述符、`VmaMapleTree` B+Tree 索引和 `VmObject` 后端对象三层架构，统一管理 ELF 段、堆、栈、`mmap` 区和共享内存附加。将缺页处理、COW、惰性分配的路径收敛到统一的 `fault_page` 入口。
- **双架构统一 virtio-blk 框架**——将 RISC-V MMIO 和 LoongArch PCI 的 virtio-blk 传输差异封装为 `transport` 适配层，通用层统一管理 `request`、`descriptor`、`completion` 和 `buffer` 回写。在此基础上建立了 `mClock` 风格 I/O 调度器，按 `nice` 值划分服务类，并在同类内按进程 `flow` 轮转。
- **网络数据面与 Socket ABI 完善**——从零构建 TCP/UDP 协议栈，支持本机 `loopback` 及外部 IPv4 访问。打通“`socket` -> `onps` 协议栈 -> `VirtIO-Net`”完整链路，使 `socket` 层具备真实 `payload` 传输能力。
- **交互式 BusyBox ash**——新增 `make shell r/1` 入口，使用独立 `ext4` `rootfs` 镜像进入 `BusyBox ash` 交互终端。打通了 `UART`→控制台行规程→`device_file`→`fd 0`→`BusyBox` 的完整输入链路，支持 `pwd`、`ls`、`cat`、脚本执行和正常 `exit`。
- **动态 ELF 与 shebang 执行链**——支持 `PT_INTERP` 动态链接器路径读取与 `musl/glibc` loader 重写，按 ELF `p_align` 对齐装载（含 LoongArch 的 16KB 对齐要求）。支持 `#! shebang` 解释器递归和参数重写，使脚本文件可直接执行。
- **系统调用与 Linux ABI 完善**——累计新增或正式接入 19 个 `syscall`，包括 `clone3`、`execveat`、`personality`、`setpriority/getpriority`、`sendmmsg/recvmmsg`、`fanotify`（`fanotify_init`、`fanotify_mark`）、`epoll_pwait2`、`memfd_create/memfd_secret`、`eventfd2`、`name_to_handle_at`、`open_tree` 等。同时修复了大量已有 `syscall` 的边界语义和 `errno` 返回值。
- **文件系统增强**——实现 `bind mount`、`open_tree` 挂载视图、虚拟文件 `/proc` 增强、`splice/sendfile` 零拷贝数据搬运、`fcntl` 文件锁修复、`flock` 完善以及 `loop` 设备支持。
- **LoongArch 稳定性提升**——修复 `ECODE` 中断判定、`TLB` 管理、`LL/SC` 原子操作、`trapframe` 映射和用户态返回窗口寄存器恢复，使 LoongArch 侧可以稳定运行 `pthread` 多线程程序和常见用户态工作负载。
- **评测与工程体系**——建立四组合 `scoreboard`，提供 `LTP runner/parser/ranker` 工具链，日志统一输出到 `logs/run/`。所有评测结果可追踪、可复现。
- **内存与启动优化**——128 MiB 低内存适配、`DTB` 动态物理内存探测、LoongArch `Split Heap` 设计、`execve` 装载路径缓存优化，显著提升批量用户程序的启动速度。

1.6 项目目录结构

F7LY-OS	
└─ Makefile	构建入口
└─ AGENTS.md / agent_docs/	Agent 规则、架构与调试文档
└─ scoreboard/ / plan_docs/	评测进度与任务计划
└─ docs/report-src/	Typst 报告源码
└─ kernel/	
└─ boot/ hal/ trap/	启动、硬件抽象与异常处理
└─ devs/	UART、控制台、virtio 等设备
└─ mem/ proc/ shm/ tm/	内存、进程、共享内存、时间
└─ fs/vfs/ lwext4/ fat32/	VFS、ext4、FAT32
└─ net/	loopback + ONPS/VirtIO-Net
└─ sys/	Linux ABI 系统调用层
└─ libs/ link/	基础库与链接脚本
└─ user/app/ user_lib/ syscall_lib/	用户态入口、测试调度与 syscall
封装	
└─ user/deps/	musl/glibc 工具链依赖
└─ images/ busybox/	QEMU 镜像与 BusyBox 产物
└─ build/ logs/	构建产物与运行日志
└─ scripts/ tools/	辅助脚本与工具
└─ thirdparty/EASTL/	内核容器库

代码 1-1 F7LY-OS 项目目录结构

1.7 2025 与 2026 能力对照

表 1-1 2025 与 2026 能力对照表

领域	2025 文档状态	2026 当前状态	变化类型
C++ 环境	宣称支持 C++ 异常	C++23 freestanding; 禁用异常与 RTTI	纠正
内存发现	固定内存布局为主	DTB 动态物理内存探测; 双架构 Split Heap	改进
用户地址空间	初步进程内存管理器	三层 VMA 架构; 统一 mmap、shm、brk、栈生命周期	重构
块 I/O	架构独立驱动	统一 virtio-blk 队列; mClock 风格 I/O 调度	新增
网络	BSD socket 协议栈框架	loopback TCP/UDP/AF_UNIX 数据面; ONPS/VirtIO-Net 出网	改进
进程管理	基础 fork/clone/exec/wait	线程支持; futex、robust futex、CLEARTRID; clone3; POSIX timer; epoll	改进
文件系统	C++ 文件系统 + FAT32	lwext4 ext4 根文件系统; VFS 统一接口; FAT32 数据盘; bind mount; loop 设备	新增
系统调用	约 60 个	243 个; 含 clone3、fanotify、memfd、splice 等	新增
用户入口	自动启动入口	自动启动入口; BusyBox ash shell; apk 包管理器	新增
工程体系	分散日志	能力追踪; 结构化日志; 可复现运行流程	新增
双架构	RISC-V 为主, LoongArch 基础	双架构对等; LoongArch 可运行多线程和复杂用户程序	改进
硬件平台	QEMU virt	QEMU virt; VisionFive 2; LS3A5000/LS2k1000 启动验证	新增

第二章 机器启动

F7LY-OS 支持 RISC-V 与 LoongArch 两条启动链路。两种架构共享统一的内核入口约定，但在固件交接方式、设备发现和中断控制器初始化等方面各自遵循本架构的硬件模型。

整个 kernel 可执行文件的入口点统一为 `_entry`，由链接脚本指定到对应架构的早期汇编入口。汇编入口只负责建立最小可运行环境，随后尽快进入 C++ 主初始化流程。

RISC-V 架构：启动流程：`OpenSBI (M-mode) → entry.S → start.cc → main()`。基于 SBI (Supervisor Binary Interface) 规范，Bootloader 工作在 M-mode，内核代码从 `entry.S` 开始运行在 S-mode。

LoongArch 架构：启动流程：`QEMU/EDK2 → entry.S → main()` (无 `start.cc` 薄层)。采用 LoongArch 原生启动方式，通过 DTB 进行硬件发现与初始化。

2.1 RISC-V 启动流程

2.1.1 entry.S

RISC-V 入口阶段只做三件事：根据 `hartid` 选择每个 CPU 独立的早期栈，保留固件传入的设备树地址，然后跳转到 `start()`。这一层不做复杂初始化，目的是尽快离开汇编环境，避免架构细节扩散到主流程。

2.1.2 start.cc

`start.cc` 是 RISC-V 在启动链中为过渡作用。它关闭早期分页、安装临时 trap 兜底入口、记录当前 `hartid`，并把设备树地址交给 `main()`。从这一点开始，RISC-V 与 LoongArch 尽量共享同一套内核初始化顺序。

2.1.3 主线：main() 四阶段

系统初始化分为四个主要阶段：

1. 基础服务初始化

```
DtbManager::init(dtb_addr);
k_printer.init();
trap_mgr.init();
trap_mgr.inithart();
plic_mgr.init();
plic_mgr.inithart();
intr_stats::k_intr_stats.init();
```

2. 进程与内存管理初始化

```
proc::k_pm.init();
mem::k_pmm.init();
mem::k_vmm.init();
mem::k_hmm.init();
shm::k_smm.init();
mem::SlabAllocator::init();
```

3. 初始进程、设备与文件系统初始化

```
k_devm.register_stdin();
k_devm.register_stdout();
k_devm.register_stderr();
tmm::k_tm.init();
syscall::k_syscall_handler.init();
proc::k_pm.user_init();
virtio_disk_init();
init_fs_table();
binit();
fileinit();
inodeinit();
fs::k_file_table.init();
vfs_ext4_init();
fs::k_vfs.dir_init();
fs::k_fifo_manager.init();
dev::LoopControlDevice::init_loop_control();
```

4. 启动调度器

```
proc::k_scheduler.init();
proc::k_scheduler.start_schedule();
```

2.2 LoongArch 启动流程

2.2.1 入口：entry.S

LoongArch 将 `start.cc` 阶段合并到 `entry.S` 中，在其中完成设置设备操作空间（DMWIN0）、设置指令数据访问空间（DMWIN1），CSR 寄存器的初始化，以及栈空间的设定，并开启 FPU 浮点数指令。保存 `$a1` 传入的 DTB 地址，`hartid` 使用 CSR_CPUID 动态读取。完成设置后直接 `bl main`。

2.2.2 main() 四阶段

系统初始化同样分为四个阶段，仅以下三处与 RISC-V 不同：

- 阶段一：`find_dtb_and_initrd()` 暴力搜索 DTB 与 `initrd`（RISC-V 由 OpenSBI 传入，无需搜索）；中断控制器为 `apic_init()` + `extioi_init()` 两级架构（RISC-V 为 PLIC 单级）。
- 阶段二：split heap —— DTB 包含多段内存时，低端区域做 buddy，高端区域整段做 heap + shm。
- 阶段三：`virtio_probe()` 先进行 PCI 总线枚举（RISC-V 走 MMIO 不需要）。
- 阶段四：与 RISC-V 一致。

```
extern "C" void main(uint64 hartid, uint64 dtb_addr)
{
    k_printer.init();
    printfYellow("Hello, World!\n");

    // Initialize DTB and scan Initrd if necessary
    uint64 kernel_end_phys = ((uint64)end) & 0x0FFFFFFFFFFFFFFFUL;
    DtbManager::find_dtb_and_initrd(dtb_addr, kernel_end_phys);

    printfMagenta("[main] Using hartid=%lu, k_dtb_addr=0x%lx\n",
        hartid, k_dtb_addr);
```

```
apic_init();
extioi_init();

// ... 与 RISC-V 阶段一至阶段三相同的初始化 ...
virtio_probe();
virtio_disk_init();
// ...
```

2.3 双架构支持特色

- 统一的抽象层：通过 HAL(硬件抽象层)实现跨架构代码复用。
- 架构特定优化：针对不同架构的特性进行专门优化。
- 模块化设计：核心模块与架构相关代码分离，便于维护和扩展。
- 现代化实现：采用 C++面向对象设计，提供类型安全和更好的代码组织。

2.3.1 启动完成标志



```
=== SYSTEM BOOT COMPLETE ===
Kernel space successfully initialized
```

第三章 中断管理器

3.1 整体设计

从用户态到内核态的切换需要中断和异常的频繁处理，此处 F7LY 在 xv6 的 trap 框架基础上选择了对象化中断管理器类 TrapManager，并实例化全局对象 trap_mgr。

```
class trap_manager
{
    friend class tmm::TimerManager;
public:
    void init();
    void inithart();
    int devintr();
    void usertrap();
    void usertrapret();
    void kerneltrap();
private:
    void timertick();
    void set_next_timeout();
    SpinLock tickslock;
    uint ticks;
    uint timeslice;
};
```

F7LY 在其底层根据 RISC-V 和 loongarch 的不同硬件设备使用而封装了不同的中断处理器，在不同文件夹下实现了 PLIC、EXTIOI、APIC 的驱动、中断统计管理器和统一的包装函数层并应用在中断处理之中。

3.2 RISC-V 中断处理路径

RISC-V 使用 **PLIC** 单层中断控制器。用户态异常从 **uservec.S** 入口，保存寄存器与 **FPU** 现场后切换内核页表，进入 **usertrap()** 统一分发；内核态异常走 **kernelvec.S** 进入 **kerneltrap()** 处理设备中断和时钟抢占。返回时由

`usertrapret()` 设置 `trapframe`，跳转 `userret` 恢复寄存器后 `sret` 返回用户态。控制台输入走 SBI，`uservec` 新增 FPU 现场保存。

3.3 LoongArch 中断处理路径

LoongArch 使用 APIC + ExtIOI 两级中断控制器。三个异常入口 `kernelvec`、`handle_tlbr`、`handle_merr` 分别由 `eentry` / `tlbrentry` / `merrentry` 三个 CSR 指定。用户态和内核态异常均从 `kernelvec.S` 入口，保存寄存器后从 `CSR_CPUID` 重取 `hartid`，进入 `usertrap()` 按 `ecode` + `esubcode` 二级编码分发；返回时由 `usertrapret()` 原子重映射 `trapframe`，经 `ertrn` 回到用户态。与 RISC-V 关键差异在于它的两级中断控制器、时钟中断 `TICLR` 直接写清除位、TLB 软件走表。

3.4 陷阱分发逻辑

`usertrap()` 进入后首先检查是否来自用户态，然后读取异常编码区分陷阱类型：

- 系统调用：
 - 检查进程是否被标记为结束。
 - 将返回地址指向 `ecall` 下一条指令。
 - 开中断，进入 `invoke_syscaller()` 查表分发。
- 缺页异常：
 - 先尝试 COW 解决共享写时复制页。
 - LA 额外检查 PTE 是否已合法，若只是 TLB 残留则做精准失效重试。
 - 进入 `mmap_handler()` 尝试惰性分配。
 - 以上均失败则发送 `SIGSEGV`。
- 地址错误：
 - 直接发送 `SIGSEGV`。
- 其他同步异常（非法指令、断点、对齐错误等）：
 - 按编码分别发送 `SIGILL`、`SIGBUS`、`SIGTRAP`、`SIGSYS` 等信号。
- FPU 禁用（LA 专属，`ecode=0xF`）：
 - 标记当前线程需要 FPU，返回用户态时懒恢复现场，不在此处保存浮点寄存器。
- 设备中断：

- 进入 `devintr()`。先确保当前为中断场景（RISC-V 检查 `scause` 高位，LA 检查 `ecode==0`），再按 `pending bit` 分发：硬件中断通过 PLIC/ExtIOI 认领并路由到设备驱动，时钟中断推进 `ticks` 并检查 POSIX 定时器。

所有分支处理完毕后，如有待处理信号则进入 `handle_signal()`。若本次为时钟中断，`timeslice` 自增并在达到阈值时调用 `yield()` 触发调度。最后通过 `usertrapret()` 返回用户态。

第四章 内存管理

4.1 内核地址空间

4.1.1 物理内存发现与布局

内核通过设备树 (DTB) 获知物理内存的分布。DTB 由固件 (OpenSBI / EDK2) 在启动时经 `$a1` 寄存器传入, `PhysicalMemoryManager::init()` 解析其中 `/memory` 节点的 `reg` 属性, 得到每段可用 RAM 的基地址与大小。

RISC-V 的 qemu virt 平台通常只有单段连续 RAM。内核以 DTB 所在地址的页对齐下界作为物理内存安全上界, 保护 DTB 不被 Buddy 分配器覆盖。可用内存按约 2:1 比例划分为低端 Buddy 页分配区和高端内核堆 + SHM 共享内存区。

LoongArch 的情况更复杂。以 `-m 1G` 为例, 物理地址空间被拆为低端 RAM (含内核镜像) 和高端 RAM (`0x90000000` 以上), 中间夹着 PCI 配置空间等空洞。PMM 首先定位包含内核 `end` 符号的 `region` 作为 Buddy 管理区, 然后从高地址向低地址搜索第一块足够大的独立 `region` 整段划给内核堆和 SHM (Split Heap)。这样低端区全留给用户页分配, 高端区独立承载内核堆, 绕开了物理地址空洞。

LoongArch 通过 DMWIN (Direct Mapping Window) 实现物理地址的直接访问, 无需页表翻译。DMWIN0 覆盖 MMIO 区域, DMWIN1 覆盖内存区域。内核提供 `to_vir` / `to_phy` / `to_io` 等地址转换宏, 释放物理页时通过 `normalize_managed_page_addr` 统一折算防止错误地址塞回 Buddy。

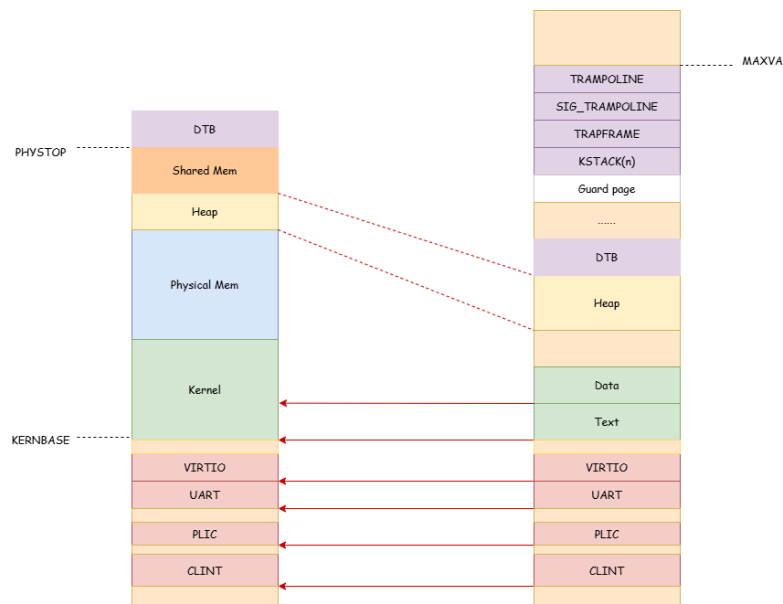


图 4-1 内核地址空间示意图

4.1.2 内核内存分配器

内核采用三层分配体系。最底层是 Buddy System:以 4KB 页为单位的伙伴分配器，内部用扁平数组实现的完全二叉树管理空闲页块，节点标记为空闲、已用、分裂或全满四种状态，分配时自顶向下二分搜索，释放时递归合并空闲伙伴。

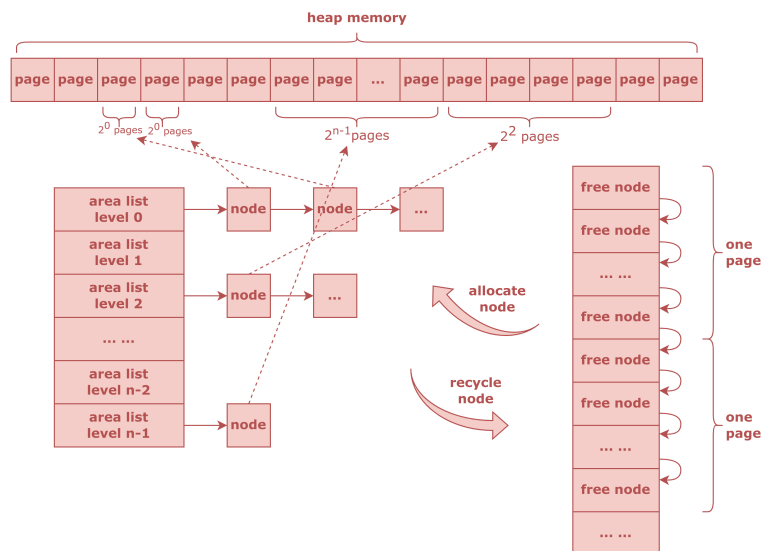


图 4-2 buddy-allocator

中间层 `PhysicalMemoryManager` (`k_pmm`) 封装 Buddy，从内核 `end` 符号处启动，预留元数据后初始化页池，对外提供 `alloc_page` / `free_page` 接口。全局

数组记录每个物理页的 16 位引用计数——`alloc_page` 初始设为 1，`fork` 时 `retain_page` 递增，`free_page` 递减到零才归还 Buddy，这是 COW 的基础。

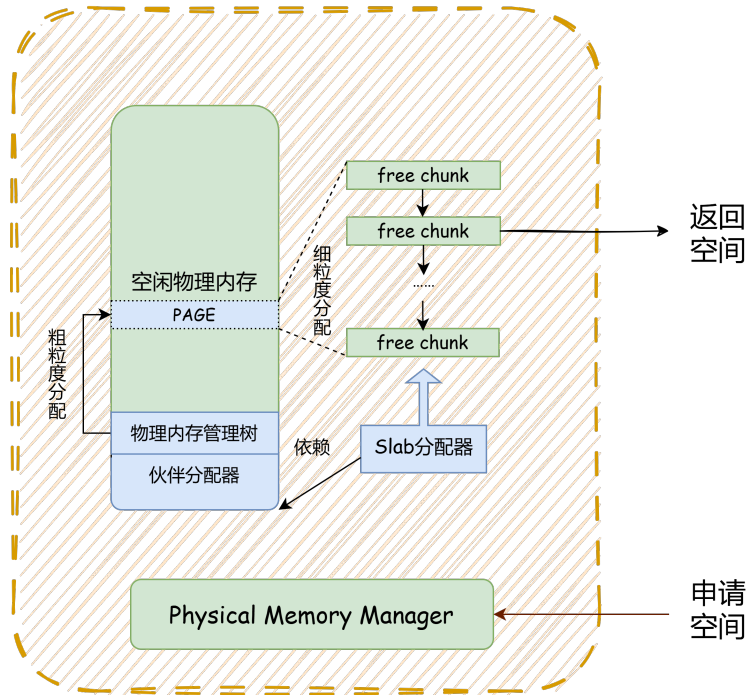


图 4-3 物理内存管理示意图

上层 `HeapMemoryManager`（`k_hmm`）解决 Buddy 只能按页分配的问题。它在内核堆区域上叠加两层架构：粗粒度层用 Buddy 申请 32 页大块，细粒度层用 `liballoc` 算法在块内做任意大小的 `malloc` / `free`。内核所有 `new` / `delete` 经重载统一路由到 `k_hmm`。

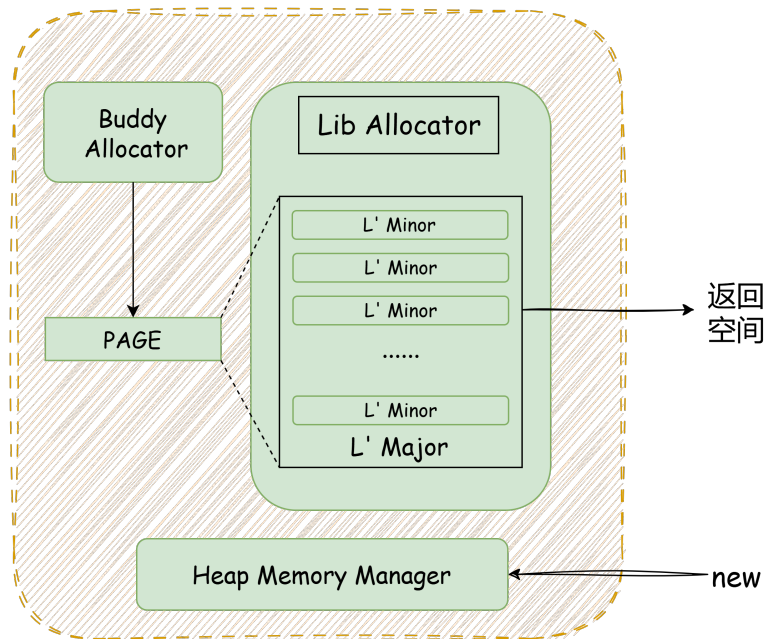


图 4-4 堆空间管理示意图

此外，`SlabAllocator` 为 16 256 字节的高频固定大小对象提供 $O(1)$ 缓存，维护空闲、部分满、全满三个链表，作为可选优化保留。

4.1.3 双架构页表差异

两个架构在内核地址空间的映射方式上有本质差异。RISC-V 采用 Sv39 三级页表的恒等映射，内核链接在 `0x80200000`，通过页表访问所有物理内存和设备 MMIO。LoongArch 凭借 DMWIN 直映窗口直接访问物理地址，内核链接在 `0x9000000080000000`，页表更多服务于用户态地址空间。

内核页表由 `kvmmake()` 创建。RISC-V 侧依次映射设备 MMIO、内核 text（只读可执行）/ data（可读写）、DTB 页面、TRAMPOLINE 跳板页和内核堆区。LoongArch 侧分情况映射低端连续区（去掉 DMWIN 掩码后的虚拟地址）和高端 Heap 区，已由 DMWIN 覆盖的区域不再重复走页表。

每个进程的内核栈位于 `TRAPFRAME` 下方，占 2 页加 1 页未映射 guard page。栈溢出触及 guard page 会触发异常而非静默破坏相邻数据。

4.2 用户地址空间

4.2.1 用户地址空间布局

F7LY-OS 的用户进程地址空间从低地址到高地址依次排布，所有区域均以 VMA 描述，并由进程内存管理器统一维护。

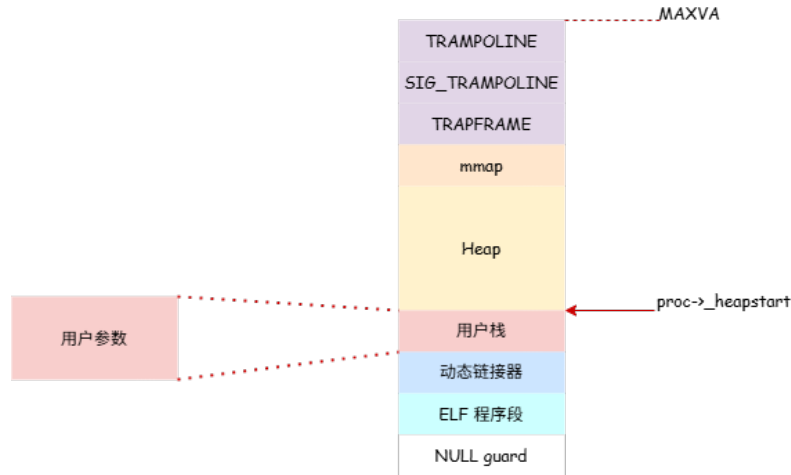


图 4-5 用户地址空间示意图

NULL guard:低 64KB（地址 `0x0 ~ 0x10000`）保持未映射状态，用于捕获空指针和 NULL 偏移解引用，触发缺页异常后向进程投递 `SIGSEGV`。

ELF 程序段:静态链接或主可执行文件的装载段按 ELF 声明的位置和权限映射。位置无关程序由内核选择装载基址，各段仍保持 ELF 头中的读、写、执行权限约束。

动态链接器段:若 ELF 声明了解释器，内核会额外装载动态链接器，并按目标架构要求对齐。主程序和解释器共同组成最终用户态入口。

用户栈:主程序和解释器装载完成后，内核在其上方建立用户栈，并在边界处保留保护页。栈支持按需向下扩展，线程栈则由线程库通过映射区单独创建。

堆:堆由 `brk` 系列接口向高地址增长。内核为堆和顶端保留区之间设置边界，避免用户堆无限扩张破坏特殊映射。

mmap 区域:堆上方是通用映射区，承载匿名映射、文件映射、共享内存附加以及线程栈。地址选择由 VMA 索引辅助完成，避免线性扫描整个稀疏地址空间。

顶端特殊页:

- **TRAMPOLINE**：内核态与用户态切换的跳板代码页，权限 `R | X`，同时映射到内核和每个用户进程的页表中。
- **SIG_TRAMPOLINE**：信号处理返回的跳板页，`sigreturn` 通过此处安全恢复被信号打断前的寄存器现场。

- **TRAPFRAME**：进程陷阱帧页面。用户态陷入内核时 `uservec` 将通用寄存器保存到此页，返回用户态时 `userret` 从此页恢复。

这三个页面由 `create_pagetable()` 在进程创建时统一映射，不注册为 VMA——它们不属于用户可控的 `mmap / munmap` 范围。

4.2.2 进程内存管理器 (ProcessMemoryManager)

`ProcessMemoryManager`（以下简称 `mm`）是一个进程全部用户态内存资源的管家。每个 `Pcb` 持有一个 `mm` 指针，`mm` 内部维护进程页表、程序段描述、`brk` 堆的起止地址（`heap_start / heap_end`）、`mmap` 自动选址用的 `mmap_cursor`，并将全部 VMA 的生命周期统一交给 `VMASpace` 管理。

```
class ProcessMemoryManager
{
public:

    // 程序段管理
    program_section_desc prog_sections[max_program_section_num];
    int prog_section_count;

    // 堆内存管理
    uint64 heap_start;
    uint64 heap_end;
    uint64 mmap_cursor;

    // 页表管理
    mem::PageTable pagetable;

    // 地址空间元数据
    VMASpace vm_space;

    // 共享标志
    bool shared_vm;

private:
    // 内存大小
    uint64 total_memory_size;
```

```

public:
    //...各种公共方法
}

```

`mm` 用引用计数区分“共享”和“独立”两种地址空间模型。`fork` 创建独立副本，并把已有物理页转为写时复制；带共享地址空间标志的 `clone` 只增加引用计数，使多个线程看到同一份堆和映射区；`execve` 则创建全新的地址空间并原子替换旧空间。最后一个引用退出时，内核按 VMA、程序段、堆和页表的顺序释放资源。

VMA 和堆的元数据操作受 `SleepLock`（`memory_lock`）保护。之所以用睡眠锁而非自旋锁，是因为 `munmap` 等操作可能触发文件映射脏页写回，进而进入 `ext4` 和 `virtio` 的磁盘 I/O 路径——自旋锁在此期间会导致死锁。

4.2.3 VMA 组织

VMA（Virtual Memory Area）是 Linux 描述进程地址空间的核心抽象——一段连续的虚拟地址区间，拥有统一的权限和同一个数据来源。F7LY 采用“描述符 → 容器与索引 → 后端对象”的三层组织方式来管理 VMA。

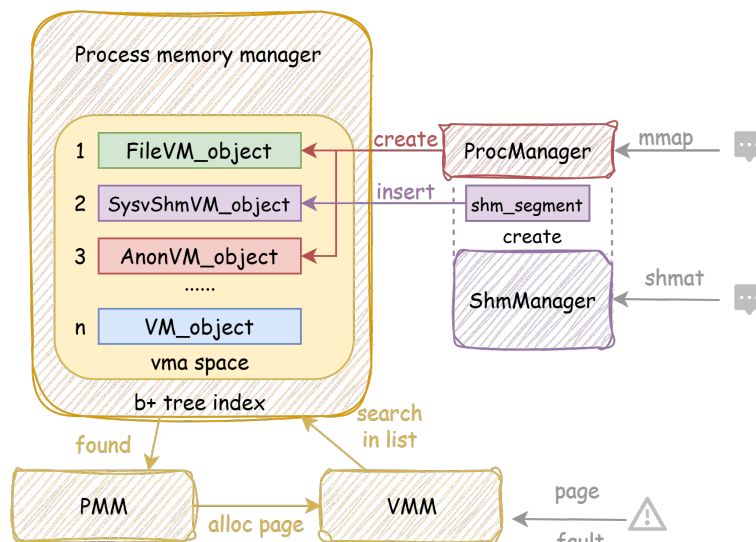


图 4-6 vma 示意图

4.2.3.1 VMA 描述符（VmArea）

一个 VMA 描述用户地址空间中一段连续区间，核心在于：

VMA 按来源分为六种类型：ELF 装载的程序段、动态链接器段、堆、用户栈、`mmap` 映射和 `SysV` 共享内存。类型决定了它的增长方式和清理顺序——比如堆只向上扩展，栈只向下扩展且在 `execve` 时不必写回。

VMA 记录了读/写/执行权限以及映射方式是私有还是共享。私有映射的写入触发 COW 拆页，只影响当前进程；共享映射的写入会最终写回文件或其他进程可见。

VMA 不自己分配物理页，而是指向一个后端对象。匿名映射的后端在访问时按需产生全零页，文件映射的后端在缺页时读磁盘，共享内存的后端从共享段取页。这样同一个文件可以被多个进程各自建立一份 VMA，而底层共享同一个文件缓存对象。

对私有文件映射或 `fork` 后的共享页，一旦某个页面被写入，COW 机制会分配一个新物理页存放私有副本。VMA 内部记录这些已拆页的页号，后续访问直接使用私有副本，不再触发缺页。

4.2.3.2 VMASpace 与 Maple Tree 索引

VMASpace 是 VMA 的集中管理容器，内部同时维护两份数据：一份 `eastl::list` 链表持有所有 VMA 对象本身，另一份 `VmaMapleTree` B+Tree 按地址对它们做索引。链表用于全量遍历，例如 `free_all_memory` 清理和 `/proc/self/maps` 导出；树索引用于按地址的精确查找和区间搜索。

`VmaMapleTree` 的设计参考了 Linux 的 Maple Tree。它是一棵纯索引树，只保存 VMA 的地址顺序，不拥有 VMA 本身。给定一个地址时，内核可以快速定位覆盖它的区间，也可以顺序遍历相邻区间。

快速区间索引对 `mmap` 同样关键。`mmap` 需要在两个已有 VMA 之间找到足够大的空隙。`VmaMapleTree` 沿叶子链表顺序扫描相邻 VMA 之间的间隔，给出第一个满足对齐和大小要求的空闲区间；整个过程只遍历实际存在的 VMA 条目，不会被稀疏地址空间中的空洞拖慢。

插入和删除时，树索引与链表双写。`mprotect` 等操作可能将一个 VMA 拆成两段或三段，`munmap` 可能从中间打洞，此时 VMASpace 先更新链表中的 VMA 对象，再同步更新树索引。部分批量操作后会调用 `rebuild_vma_index()` 从链表全量重建树索引，用一次性重建替代多次碎片化更新带来的平衡开销。

4.2.3.3 VMObject 后端抽象

`VmObject` 的引入将这些数据来源的差异统一到一个抽象基类背后，对外只暴露一个接口：给定 VMA 区间内的某个页序号和访问类型（读/写/执行），返回一个可安装到页表中的物理页。

`VmObject` 有三个子类，各自实现不同的 `prepare_page` 逻辑：

匿名对象（`AnonVmObject`）：用于 `MAP_ANONYMOUS`、堆和栈。首次访问时分配一个全零物理页，再次访问同一页时直接返回已分配的页。`fork` 后父子共享同一个匿名对象，各自的 VMA 通过 COW 机制隔离写入。

文件对象（`FileVmObject`）：用于 `mmap` 文件映射。缺页时从磁盘读取文件内容到物理页中。对于 `MAP_SHARED` 的文件映射，相同文件在同一内核内共享同一个 `FileVmObject`（通过文件路径的缓存键去重），使得多个进程的共享文件映射真正看到同一份物理页，一方的写入对另一方立即可见。`MAP_PRIVATE` 的文件映射则各自持有独立的对象实例，写入触发 COW 拆页后记录到 VMA 的私有页表中，不影响磁盘原文件和其他进程。进程退出或 `munmap` 时，共享对象的脏页通过 `sync_area_range` 写回磁盘。

SysV 共享内存对象（`SysvShmVmObject`）：用于 `shmat` 附加的共享内存段。内部持有段的元数据（key、大小、权限、创建者等），`prepare_page` 从共享段的物理页数组中取页。与文件对象类似，多个进程附加同一段时共享同一个对象实例。

`VmObject` 会缓存已经准备好的源页。这样在 `fork` 后，子进程首次读取共享区域时可以复用父进程已经生成的物理页，只在写入时通过 COW 分离。

对象的生命周期由原子引用计数管理。VMA 创建时 `get()`，销毁时 `put()`。当所有引用释放后，匿名对象和 SysV 对象直接释放缓存的物理页，文件对象则先写回脏页再释放。

4.2.4 缺页异常处理

缺页异常是用户地址空间中惰性分配和 COW 的驱动入口。当用户态访问一个没有合法页表映射的虚拟地址时，硬件触发异常，经 `usertrap` 分发到 `mmap_handler`，最终由 `ProcessMemoryManager::fault_page` 按照一个固定的优先级链处理。

COW 优先:缺页后第一步是检查该地址是否命中了 COW 页面——即 PTE 存在且带有 `PTE_COW` 标志，且当前访问是写入。如果是，调用 `resolve_cow_page` 分配一个新物理页，将原页内容复制过来，把新旧 PTE 都恢复为可写并清除 COW 标

记。这一步必须在 VMA 查找之前完成，因为 COW 页在页表中已经有一个“只读 + COW”的 PTE，如果直接去查 VMA 做惰性分配会误判为未映射。

惰性分配:如果 PTE 根本不存在（真正的缺页），通过 `VMASpace` 查找覆盖该地址的 VMA。找到后，检查访问类型是否与 VMA 声明的权限匹配——例如对只读映射写入会直接返回失败。权限检查通过后，调用 VMA 绑定的 `VmObject::prepare_page` 获取物理页，再根据 VMA 的 `prot` 和 `flags` 构建 PTE 标志位（私有映射写入时对匿名/文件对象标记 COW，共享映射写入时直接标可写），安装到页表中。

栈自动扩展:如果缺页地址没有命中任何 VMA，但紧邻用户栈 VMA 的下方（栈向下增长方向），且地址高于栈的 `guard page` 上界，则内核自动将栈 VMA 的起始地址向下扩展到缺页地址，然后重新走惰性分配流程。这种自动扩展使得 `MAP_GROWSDOWN` 栈可以按需增大，而非一次性分配全部 `max_len` 的物理页。

TLB 残留重试(仅 LoongArch):LoongArch 的 TLB 由软件管理，可能出现 PTE 已正确安装但 TLB 中仍残留无效表项的情况。在进入 VMA 查找之前，若 PTE 已存在、有效且权限与当前访问匹配，内核执行一次 `invtlb` 精准失效后直接返回重试用户指令，不将其误判为缺页失败。

以上任一环节失败，进程收到 `SIGSEGV`，内核在信号帧中记录 `si_addr` 为触发异常的虚拟地址，供用户态调试器或信号处理函数定位。

第五章 进程与线程管理

5.1 进程控制块 (PCB)

进程控制块 (Process Control Block, PCB) 是内核管理进程与线程的核心数据结构。F7LY 以 `Pcb` 统一描述一个任务的全部内核态状态，将标识、调度、地址空间、文件系统、信号处理、同步原语、资源限制和时间统计等信息收拢到同一个对象中。

进程与线程在 F7LY 中统一用 `Pcb` 表示，区别在于资源共享关系：线程共享地址空间和线程组，但仍拥有独立的内核栈、陷阱帧和线程 ID。内核通过线程 ID 与线程组 ID 的关系区分主线程和普通线程。以下两小节分别展开 PCB 的组成和状态机。

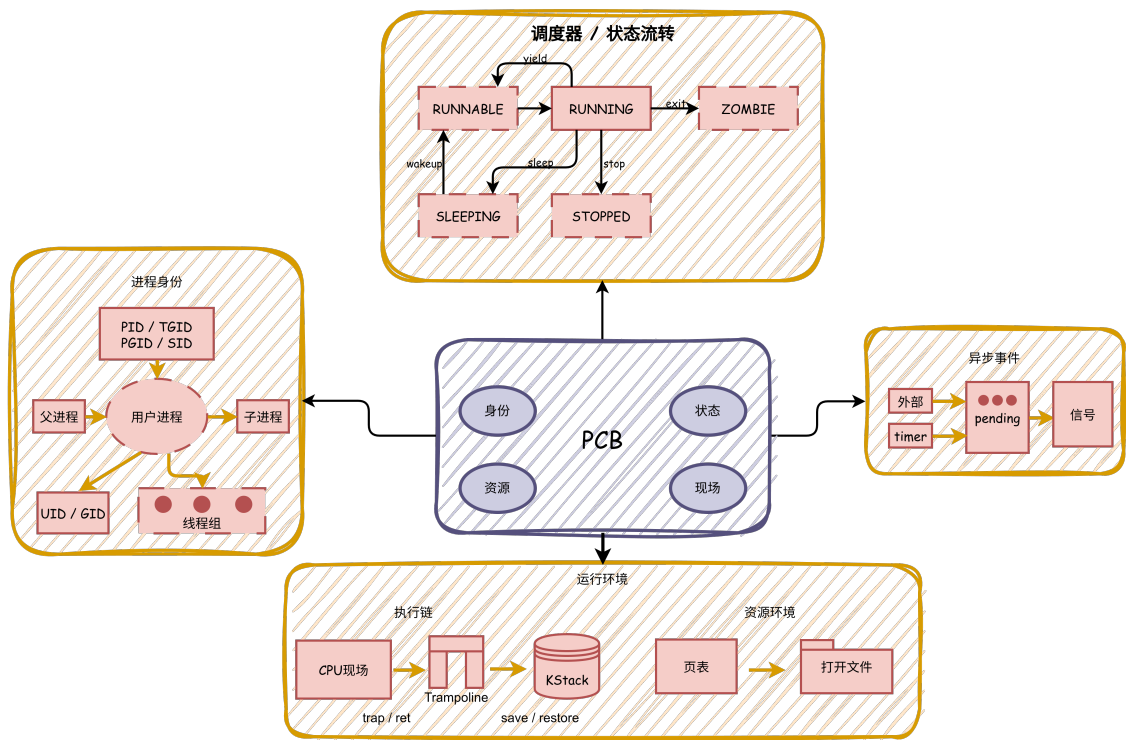


图 5-1 PCB 结构示意图

5.1.1 组成

```
class Pcb;
extern Pcb k_proc_pool[num_process];
```

Pcb 是进程控制块的类型名，k_proc_pool 是全局进程池，以静态数组形式容纳系统中所有进程与线程的 PCB。每个 Pcb 对象承载一个任务的完整上下文，详细字段放于文末附录进行介绍。

5.1.2 状态机

进程在其生命周期中会处于不同的状态。F7LY 使用七种状态描述任务从空闲、分配、就绪、运行、阻塞、停止到退出待回收的完整过程。

表 5-1 进程七状态说明

状态	含义
UNUSED	进程池槽位空闲，PCB 尚未被分配。
USED	PCB 已被分配但尚未就绪调度，正在初始化运行所需的资源。
RUNNABLE	进程准备就绪，等待调度器分配 CPU。
RUNNING	进程正在 CPU 上执行。
SLEEPING	进程因等待某个资源或事件而阻塞，条件满足后被唤醒回到就绪状态。
STOPPED	进程被 job-control 停止信号暂停，收到继续信号后恢复就绪。
ZOMBIE	进程已终止，但 PCB 保留退出状态等待父进程回收。

进程状态的切换如下：

- 进程池创建时，所有 PCB 初始化为 UNUSED。
- 创建新进程时，从池中取出空闲 PCB，分配 PID、trapframe 和内核栈后，切换为 USED。
- 初始化完成、具备运行条件后，切换为 RUNNABLE，进入调度器的就绪队列。
- 调度器选中该进程后，状态变为 RUNNING，进程开始执行。
- 时间片耗尽或进程主动让出 CPU 时，状态回到 RUNNABLE。
- 进程等待资源或事件时（如管道读写、futex 等待），状态变为 SLEEPING；条件满足后被唤醒，回到 RUNNABLE。
- 进程收到停止信号（SIGSTOP、SIGTSTP 等）时，状态变为 STOPPED；收到 SIGCONT 后恢复为 RUNNABLE。

- 独立进程或线程组主线程退出时，状态变为 `ZOMBIE`，保留退出码等待父进程回收；普通线程完成清理后直接归还 `PCB`。

5.2 进程调度

调度器负责决定下一个在 `CPU` 上运行的任务。F7LY 的调度建立在三个机制之上：上下文切换完成执行现场保存与恢复，轮转策略保证就绪任务公平获得 `CPU` 时间，优先级策略让高优先级任务更早被选中。以下三小节依次展开。

5.2.1 上下文切换

上下文切换是调度器在进程之间切换 `CPU` 的底层机制。当一个进程让出 `CPU`、另一个进程被选中时，内核需要保存当前进程的现场，并恢复目标进程之前保存的现场——这样目标进程从上次停下的位置继续执行，就像从未离开 `CPU`。

保存和恢复的现场由内核态上下文记录，主要包括栈指针、返回地址以及被调用者保存寄存器。实际切换由架构汇编完成：当前任务切出时保存自己的上下文并回到调度器，目标任务切入时从已保存的上下文继续执行。

一次完整调度由“任务切出到调度器”和“调度器切入另一个任务”两个动作组成。两次切换形成镜像，使每个任务再次被调度时都能从上一次停下的位置继续执行。

5.2.2 轮转调度

轮转调度是调度器的基本策略：调度循环按进程池顺序扫描可运行任务，运行任务在时间片耗尽或主动让出时回到就绪态，从而让就绪任务持续获得执行机会。

调度循环每轮先计算当前最高有效优先级，再从进程池中选择状态为 `RUNNABLE` 且优先级匹配的任务执行。任务被选中后状态切为运行态；当它因为时间片耗尽、主动让出或等待资源而切出时，调度器继续寻找下一个就绪任务。

时间片的计量依赖时钟中断。每次 `timer tick` 到来时，中断处理程序检查当前进程是否已用完本轮时间片；若已用完，则触发 `yield` 将进程状态从 `RUNNING` 改回 `RUNNABLE`，并调用 `swtch` 切回调度器，让下一个进程上场。

5.2.3 优先级调度

在轮转调度的基础上，调度器引入优先级机制，使高优先级进程优先获得 CPU。

每个进程有一个优先级数值（`nice` 值，范围 `[-20, 19]`，越小优先级越高），默认值为 `0`。`nice` 值不仅影响 CPU 调度，也同时影响块层 I/O 的排队顺序——计算密集型和 I/O 密集型的优先级策略在此统一。进程还可以单独设置 I/O 优先级覆盖值，在保持 CPU 调度不变的前提下调整磁盘访问的优先顺序。

调度器在每轮扫描进程池时，先遍历所有进程找出当前就绪进程中的最高优先级（即最小 `nice` 值），然后以该数值作为门槛：只选择优先级等于该值的进程执行。这保证了只要存在高优先级的就绪进程，低优先级进程就不会被选中。当所有进程的优先级都是默认值 `0` 时，调度器通过一个布尔标记跳过完整的优先级统计，直接退化为纯轮转——这是最常见的场景，几乎不引入额外开销。

此外，调度器在计算优先级时会将信号状态纳入考虑：当进程有未屏蔽的信号等待处理时，该进程在当轮调度中会被临时视为最高优先级（`-20`）。这确保信号处理函数能尽快在用户态执行，而不必排在与它同优先级的其他进程之后。这种提升只影响调度器内部的当轮决策，不改变进程本身的 `nice` 值。

5.3 进程生命周期

进程的生命周期涵盖创建、执行与退出三个阶段。F7LY 通过 `fork / clone` 创建新任务，通过 `execve` 装载新程序映像，通过 `exit / wait` 终止与回收。此外，`clone` 支持创建共享地址空间的线程，使多线程编程成为可能。以下四小节依次展开。

5.3.1 创建（`fork / clone / clone3`）

新任务的创建由 `fork` 和 `clone` 两个系统调用完成。`fork` 是传统的进程复制语义——创建一个与父进程几乎完全相同的子进程。`clone` 在此基础上增加了标志位控制，允许调用者精细决定哪些资源在父子间共享、哪些独立复制，从而既能创建进程，也能创建线程。`clone3` 是 `clone` 的扩展版本，提供了更大的参数空间和更整洁的接口。

创建过程的核心步骤相同：首先从全局进程池中分配一个空闲 PCB，为其分配独立的 PID 和 TID，创建专用的 `trapframe` 和内核栈；然后根据调用类型决定地址

空间、文件描述符表和信号处理表的处理方式——`fork` 将这些资源深拷贝给子进程，`clone` 则根据标志位选择性地共享。

子进程首次被调度时，从其内核栈上恢复上下文，并从 `fork` 的返回点开始执行。`fork` 的返回值是区分父子进程的关键：父进程收到子进程的 PID，子进程收到 0。`clone` 在线程模式下返回新线程的 TID，在进程模式下返回新进程的 PID。

`clone` 的关键标志位决定父子之间共享哪些资源。线程创建通常共享地址空间、文件描述符表和信号处理表，并加入同一个线程组；进程创建则保留独立资源。另一些标志负责 `vfork` 阻塞、线程局部存储以及线程退出时的清零和唤醒语义，为用户态线程库提供底层支撑。

5.3.2 装载 (`execve`)

`execve` 的功能是用一个新的程序映像替换当前进程的地址空间。与 `fork` 不同，它不创建新进程，而是在当前进程中销毁旧的代码、数据和栈，加载新的可执行文件并从头开始执行。进程的 PID、文件描述符表（除标记了 `CLOEXEC` 的项外）和信号屏蔽字等属性保持不变。

装载过程的核心是解析 ELF 可执行文件。内核读取 ELF 文件头，遍历程序头表中的 `PT_LOAD` 段，将每个段加载到指定的虚拟地址，并按其声明的权限（读、写、执行）进行映射。如果 ELF 包含 `PT_INTERP` 头——即程序是动态链接的——内核还需额外加载动态链接器（如 `/lib/ld-musl-riscv64-sf.so.1`）的各 `LOAD` 段。最终的程序入口点设为可执行文件自身的入口地址，或者动态链接器的入口地址。

对于以 `#!` 开头的脚本文件，内核会识别 `shebang` 行，提取解释器路径和可选参数，递归地以解释器为目标重新启动装载流程。这意味着用户执行脚本时，实际装载的是脚本解释器，脚本路径作为参数传入。

新地址空间构建完成后，内核构造用户栈，依次压入参数和环境变量的字符串、指针数组、以及辅助向量（auxiliary vector），然后原子地替换当前进程的页表，释放旧地址空间。原来注册的信号处理函数中，除设置为忽略（`SIG_IGN`）的以外全部重置为默认行为，信号备用栈也一并清除。最后，将程序计数器设为 ELF 入口地址，栈指针指向新构造的用户栈顶，进程从新程序的第一条指令开始执行。

5.3.3 退出 (`exit` / `wait`)

进程的退出有两种粒度：`exit` 仅终止当前线程，`exit_group` 终止整个线程组（即进程的所有线程）。无论哪种方式，最终都进入 `exit_proc` 执行清理。

清理过程分两个阶段。首先完成所有必须在地址空间还存活时进行的用户态可见操作：将 `CLONE_CHILD_CLEAR_TID` 指定的地址清零并执行 `futex` 唤醒（使 `pthread_join` 得以返回），遍历健壮 `futex` 链表对持有的互斥锁标记“持有者已死”并唤醒等待者。然后逐步释放进程持有的内核资源——关闭所有文件描述符、释放信号处理表、销毁地址空间（包括 `VMA`、堆内存和页表）。

资源释放完成后，进入状态终结。普通线程直接归还 `PCB`，不产生僵尸态；主线程退出时代表整个进程终止，需要保留退出状态、通知父进程，并把遗留子进程交给 `init` 接管。

父进程通过 `wait4` 或 `waitid` 等待子进程状态变更。当子进程退出变为 `ZOMBIE` 时，内核将子进程的退出码和运行时间信息返回给父进程，然后回收 `PCB`。如果父进程明确将 `SIGCHLD` 设置为忽略，或设置了 `SA_NOCLDWAIT` 标志，则子进程退出时不再保留 `ZOMBIE`，直接自动回收。

5.3.4 线程

线程是同一进程内的独立执行流。F7LY 中线程与进程统一使用 `Pcb` 表示，区别在于 `clone` 时传入的标志位组合。线程的完整生命周期如下：

创建——用户态线程库（如 `musl` 的 `pthread_create`）调用 `clone` 系统调用，传入 `CLONE_VM | CLONE_THREAD | CLONE_FILES | CLONE_SIGHAND` 等标志。内核分配新的 `PCB`、`PID` 和 `TID`，但共享父进程的地址空间（`_memory_manager` 引用计数加一）、文件描述符表和信号处理表。新线程拥有独立的内核栈和 `TrapFrame`。`CLONE_SETTLS` 将线程局部存储的基址写入 `tp` 寄存器，`CLONE_CHILD_CLEAR_TID` 记录一个用户态地址供退出时使用。

标识——新线程继承父进程的线程组标识，同时获得独立线程号。主线程和普通线程的区分由这两个标识之间的关系决定。

正常退出——线程调用 `exit` 退出时，先清理用户态线程库约定的退出地址并唤醒等待者，使 `join` 方能够返回。内核随后释放线程持有的资源；普通线程直接回收 `PCB`，不产生僵尸态。

异常退出——如果线程异常终止（如被信号杀死），内核在清理时遍历线程注册的健壮 `futex` 链表。对链表中每个被该线程持有的互斥锁，将其标记为

`FUTEX_OWNER_DIED`（持有者已死），并唤醒在该锁上等待的其他线程。这防止了线程异常终止导致的死锁——等待者可以检测到锁的异常状态并采取恢复措施。

主线程退出——主线程退出时会终止整个线程组。内核标记同组线程结束，各线程在再次进入内核或被调度时完成清理；主线程自身进入僵尸态，等待父进程回收。

第六章 文件系统

6.1 设备抽象层

F7LY 的设备层目标是把“硬件差异”和“文件接口”隔离开。字符设备和传统块设备通过 `VirtualDevice / DeviceManager` 暴露设备类型、中断处理和读写就绪状态；`virtio-blk` 走独立的跨架构队列框架；`loop` 设备由 `/dev/loop-control` 管理实例，并由 `/dev/loopN` 虚拟设备节点转发到后端文件。它们对上最终都收束为 VFS `fd` 或设备节点，使 VFS、`epoll` 和系统调用层主要关心 `fd` 是否可读写。

字符设备面向字节流，承担控制台、串口、伪终端等交互式 I/O。流设备在字符设备之上补齐批量读写能力，使 VFS 可以用普通文件的方式处理终端输入输出。传统块设备面向固定大小的数据块，服务 `ramdisk` 等内存块后端，并支持一次请求携带多个缓冲区；`virtio-blk` 的批量传输由独立队列层承担。

`Loop` 设备把普通文件包装成可被块语义访问的对象。控制面通过 `/dev/loop-control` 创建和管理实例，数据面通过 `/dev/loopN` 的虚拟设备节点把块读写映射到后端文件的对应偏移。这样文件系统镜像可以像真实磁盘分区一样被挂载，同时后端文件的生命周期由 `loop` 设备独立持有，不受原始 `fd` 关闭影响。

6.2 块设备驱动与 I/O 调度

6.2.1 统一 `virtio-blk` 框架

F7LY 的 `virtio-blk` 驱动采用三层框架，将硬件传输、请求队列和块设备接口拆开。RISC-V 使用 MMIO，LoongArch 使用 PCI，但这种差异只存在于传输层；队列层和设备层共享同一套请求管理逻辑。

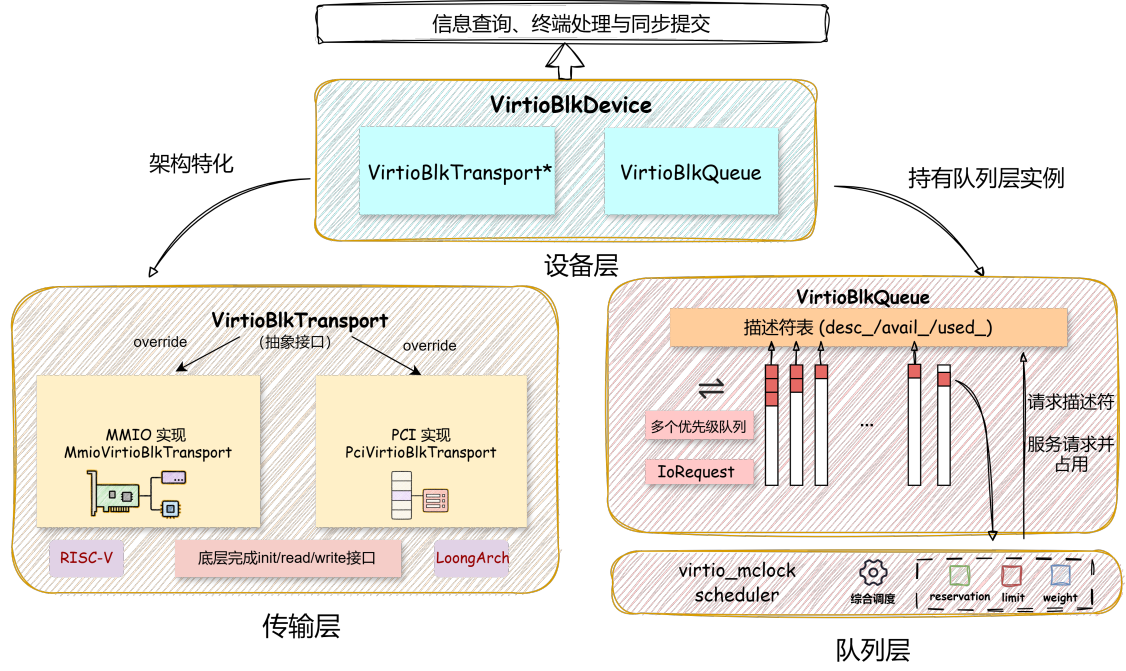


图 6-1 块设备驱动与 IO 调度

传输层负责 DMA 地址转换、队列通知和中断确认。队列层负责描述符分配、请求排序、完成回收和并发控制。设备层向上提供同步提交接口，把 buffer cache、ext4、FAT32、loop 等来源的请求统一转换成扇区读写。请求进入队列时携带提交进程和优先级信息，使 CPU 调度和磁盘调度共享同一套优先级语义。

6.2.2 mClock 调度器

mClock 调度器位于请求进入 virtqueue 之前，用于决定多个进程并发 I/O 的派发顺序。F7LY 将进程 nice 值映射到若干服务类，每个服务类内部再按进程维护独立 flow，避免单个进程独占同一优先级的磁盘带宽。

$$R_i^r = \max \left\{ R_i^{\{r-1\}} + \frac{1}{r_i}, t \right\} \quad (6-1)$$

$$L_i^r = \max \left\{ L_i^{\{r-1\}} + \frac{1}{l_i}, t \right\} \quad (6-2)$$

$$P_i^r = \max \left\{ P_i^{\{r-1\}} + \frac{1}{w_i}, t \right\} \quad (6-3)$$

调度器同时考虑三类约束：保底带宽保证低频请求不会长期饥饿，权重标签决定空闲带宽如何分配，上限标签为后续限速预留接口。磁盘实际吞吐会通过完成路

径动态估计，调度器据此调整标签推进速度。只有一个活跃类别时，调度器直接走快速轮转路径，减少不必要的扫描。

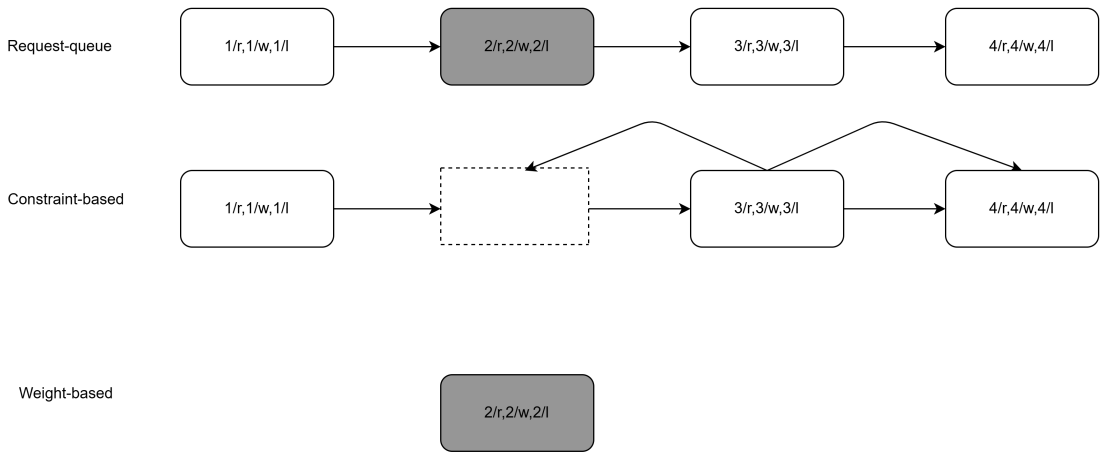


图 6-2 Reservation 标签调整

6.3 VFS 与文件对象

VFS 在 F7LY 中承上启下。对上，文件相关系统调用最终都落到统一的 `file` 对象；对下，VFS 分发到 `ext4`、`FAT32`、虚拟文件、设备文件、管道、`socket` 和 `epoll` 等不同后端。用户态看到的是同一套 `fd` 语义，内核内部则由多态文件对象承接差异。

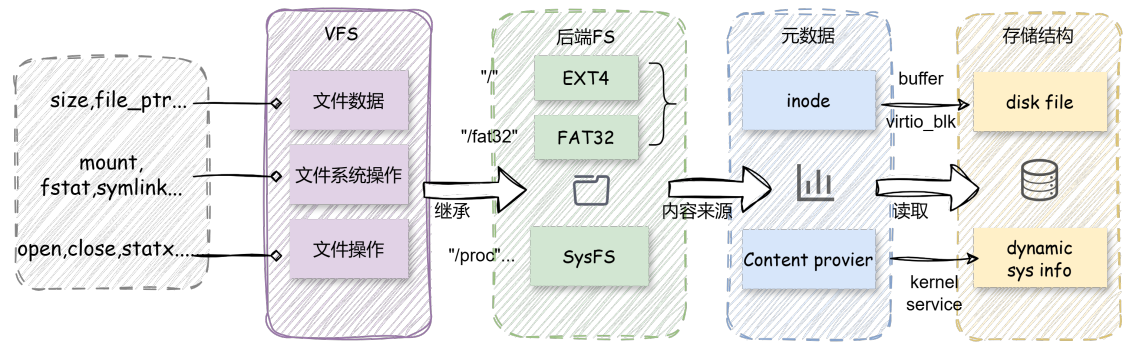


图 6-3 vfs_structure

路径打开时，VFS 先判断是否命中内存中的虚拟文件树，其次识别设备路径，剩余路径进入磁盘文件系统解析。解析成功后，内核根据资源类型创建对应文件对象，并把它放入进程 `fd` 表。读写、`seek`、目录遍历、就绪判断等操作都通过文件对象转发到底层后端。

F7LY 的文件对象体系可以概括为七类：普通文件和目录对接 `ext4/FAT32`；设备文件和虚拟设备节点对接设备管理器或虚拟文件 `provider`；管道文件对接内核循

环缓冲区；socket 文件对接网络数据面；虚拟文件按需生成内核信息；epoll 文件维护关注 fd 集合并统一查询就绪状态。

6.4 VFS 核心元数据

6.4.1 Buffer 层

Buffer 层位于文件系统和磁盘驱动之间，负责缓存磁盘块、合并写回并提供块级同步点。ext4 主要通过 lwext4 的块缓存访问磁盘，缓存未命中时再下探到底层 buffer 和 virtio-blk。FAT32 也使用同一块设备通路，因此两套文件系统最终共享相同的磁盘队列和调度策略。

为提高连续块读写和高频小写入的吞吐，F7LY 在两处做了聚合。其一，ext4 向下提交相邻逻辑块时，块设备适配层会把连续块合并成更大的 DMA 传输，减少 virtqueue 描述符分配、队列通知和中断回收次数。其二，普通文件写路径在进入 ext4 前设置 write-combine 缓冲区，把密集小写入合并后再提交，降低文件系统事务和锁竞争开销。

6.4.2 SuperBlock、Inode 与 File

VFS 元数据分为三层，每层只负责自己必须表达的抽象。

- **SuperBlock** 表示一个已挂载的持久化文件系统，保存块大小、文件系统类型、根 inode 和脏元数据回写入口。ext4 的磁盘布局细节仍由 lwext4 的超级块结构维护，VFS 只抽取跨文件系统通用的信息。虚拟文件系统不使用超级块，它直接由内存目录树描述。
- **Inode** 表示文件系统对象，记录文件类型、权限、大小、时间戳和底层文件系统私有信息。ext4 通过 extent 和目录索引定位数据，FAT32 通过簇链定位数据，但 VFS 上层只通过 inode 操作表访问。
- **File** 表示一次打开后的运行时状态，保存文件偏移、打开标志、引用关系和后端对象。两个 fd 可以指向同一个 inode，但拥有不同的 file 状态；pipe、socket、epoll 等非磁盘对象也通过 file 接入同一 fd 体系。

这三层共同贡献了 VFS 的主要抽象边界：superblock 管文件系统实例，inode 管目录树对象，file 管打开后的会话状态。系统调用层不直接操作 ext4 或 FAT32 的内部结构，而是经由这三层完成分发。

6.5 ext4 与 FAT32

F7LY 同时支持 ext4 与 FAT32。ext4 是主力根文件系统，承载系统目录、用户程序、配置文件和动态链接器；FAT32 作为辅助数据盘，服务数据交换和兼容场景。两套文件系统在磁盘布局和目录查找方式上差异很大，但在 VFS 之上暴露相同的文件对象和 inode 操作。

ext4 通过 `lwext4` 提供 extent、日志、目录索引和扩展属性支持；FAT32 使用内核内置实现维护 BPB、FSINFO 和簇链。二者的块读写最终都收口到统一 `virtio-blk` 框架，因此底层磁盘驱动不感知请求来自哪一种文件系统。

6.6 文件操作

除基本读写外，F7LY 在文件描述符层提供若干高级能力：

- `fcntl` 管理 `fd` 属性、非阻塞状态、文件锁、管道容量和 `memfd` 密封。
- `ioctl` 作为设备控制通道，按块设备、终端、`socket` 等类型拆分处理。
- 扩展属性为文件提供独立于正文数据的键值存储。
- `splice`、`sendfile` 和 `copy_file_range` 在内核内搬运数据，减少用户态往返拷贝。
- `fanotify` 和 `inotify` 为文件访问、修改和目录事件提供通知入口。

这些能力都以 `fd` 为统一入口，真正的差异由对应文件对象或设备后端处理。

6.7 挂载与路径解析

挂载系统负责把不同设备和文件系统接入统一目录树。F7LY 支持常规挂载、`bind` 挂载和挂载点移动，并维护私有、共享、从属等传播属性。每个进程属于一个 `mount namespace`，`namespace` 内的挂载变更不会直接影响其他 `namespace`，最后一个持有者退出后对应视图自动释放。

路径解析先完成语法规范化，再沿当前 `mount namespace` 的目录树逐级解析。解析过程中会区分虚拟路径、设备路径和磁盘路径，并处理符号链接、相对目录 `fd`、根目录限制等语义。最终返回的不是底层文件系统私有对象，而是 VFS 可操作的 `inode` 或 `file`。

6.8 虚拟文件与特殊设备

F7LY 的 VFS 还维护一组不依赖磁盘的虚拟文件和特殊设备。它们在用户态表现为普通路径，但数据来自内核状态、配置表、硬件设备或另一个文件对象。正文只按目录说明职责，具体节点由初始化注册表集中维护。

- `/proc` 是进程与系统运行状态窗口，提供进程自身信息、内存映射、挂载视图、内存统计、CPU 信息、中断统计和 System V IPC 状态等动态内容。
- `/proc/sys` 是可读写内核参数视图，用于暴露进程号上限、共享内存限制、管道限制、随机熵、网络标签等可配置状态。
- `/sys` 提供 CPU 和 NUMA 拓扑信息，使用户态工具可以查询在线 CPU、可用节点和节点距离等硬件视图。
- `/etc` 由内核内置配置文件补齐基础运行环境，例如用户组、主机名解析、动态链接器、DNS、协议号和时区信息。
- `/boot` 暴露内核配置文件节点，供用户态工具读取当前内核能力。
- `/dev` 注册字符设备、块设备、loop 设备、伪终端、随机数设备、实时时钟和网络隧道等特殊入口。

这些路径的共同点是：它们都复用 VFS 的 `open`、`read`、`write` 和 `close` 语义，但数据来源不一定是磁盘。VFS 因此成为普通文件、内核状态和设备控制之间的统一边界。

第七章 进程间通信

进程间通信 (Inter-Process Communication, IPC) 是操作系统向用户态程序提供的跨进程数据交换与事件通知机制。F7LY-OS 的 IPC 体系按照两条线组织：信号、POSIX Timer、eventfd 和 epoll 解决事件通知；futex、管道、FIFO、共享内存和 memfd 解决同步与数据交换。

7.1 信号与定时器

信号是异步事件通知机制。F7LY 的信号子系统围绕四类状态运转：每个任务的待处理信号集合、信号屏蔽字、信号处理动作表，以及进入用户态 handler 前保存的信号帧。只有这几类状态会影响信号语义，具体函数如何拆分不在正文展开。

信号处理路径可以概括为三个阶段。首先，内核或用户进程产生信号，将目标任务的待处理集合置位，必要时唤醒正在等待信号的任务。其次，任务从内核态返回用户态前，内核筛选未被屏蔽的信号，并根据注册动作决定处理方式。若用户注册了 handler，内核在用户栈或备用信号栈上保存被打断现场，改写返回上下文，使任务回到用户态后先执行 handler；若信号采用默认动作，则内核直接执行终止、停止或忽略。最后，handler 通过 `rt_sigreturn` 返回内核，内核恢复原始寄存器和信号掩码，用户程序继续从被打断的位置运行。

默认动作按 POSIX 语义处理：停止类信号使任务进入暂停状态，继续信号恢复调度，终止类信号进入退出路径，忽略类信号只从待处理集合中清除。实时信号和带数据的信号在 pending 位之外保留一份附加信息，使用户态能够读取发送方和携带值。

POSIX Timer 与 interval timer 都以信号作为到期通知。Interval timer 嵌在进程状态中，分别描述真实时间、用户态 CPU 时间以及用户态加内核态 CPU 时间。POSIX Timer 由全局池管理，支持用户指定时钟类型、通知目标和信号编号。每次时钟中断推进时间后，内核检查到期项，把对应信号加入目标任务的待处理集合；后续处理仍复用统一的信号返回路径。

7.2 futex

`futex` 是 Linux 提供了一种轻量级同步原语。它的核心思想是：将互斥锁的大部分操作放在用户态完成——线程在无竞争时仅通过一次原子比较交换就能获取或释放锁，完全不需要陷入内核。只有在线程需要等待（锁已被别人持有）或需要唤醒等待者时，才通过 `futex` 系统调用进入内核。这种设计使得 `futex` 在低竞争场景下几乎等同于纯用户态的原子操作，同时又能处理复杂的阻塞与唤醒逻辑。

F7LY 实现了 `futex` 的核心语义，包括 `FUTEX_WAIT` 与 `FUTEX_WAKE` 的基本 `wait/wakeup`、健壮链表（robust list）的退出自动解锁，以及 `FUTEX_REQUEUE` 的等待者迁移。

7.2.1 wait 与 wakeup

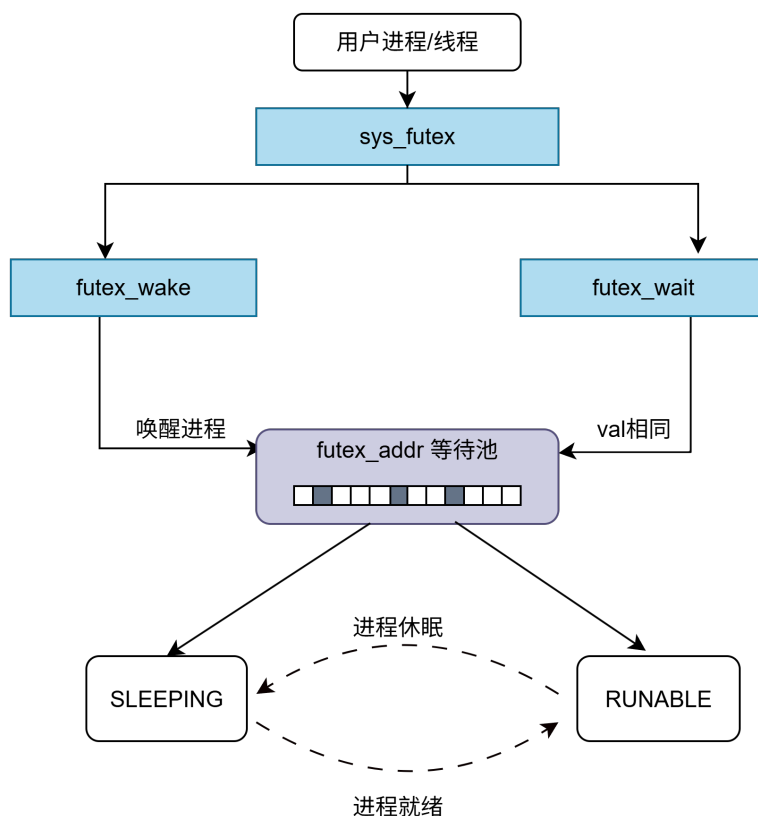


图 7-1 futex 工作机制

`WAIT` 的核心流程是先重新读取用户态地址的值，并与调用者期望值比较。如果不一致，说明状态已在进入内核前变化，内核立即返回，让用户态重新尝试；如果一致，内核才把线程放入等待队列并睡眠。这个二次比较保证了“检查条件”和“进入睡眠”之间不会丢失唤醒。

`futex` 的等待键使用用户地址对应的物理页身份，而不是裸虚拟地址。这样多个进程通过共享映射看到不同虚拟地址时，仍能落到同一个等待队列。`WAKE` 解析出同一把键后，唤醒指定数量的等待者；被唤醒线程回到用户态后重新执行原子获取逻辑。

等待过程支持相对/绝对超时，也支持被未屏蔽信号打断。超时返回超时错误，信号打断返回中断错误，用户态线程库据此决定重试、返回或进入信号处理。

7.2.2 Robust List 与 Requeue

`Robust list` 用来处理“线程持锁退出”的情况。线程库在持有健壮互斥锁时把锁登记到自己的 `robust list`，释放时摘除。线程退出时，内核遍历该链表，把仍由它持有的锁标记为“持有者已死亡”，并唤醒等待者。等待者看到该标记后可以修复互斥锁状态，避免永久死锁。

`FUTEX_REQUEUE` 面向条件变量场景。它将“唤醒少量等待者”和“把剩余等待者迁移到另一把锁”合并为一个原子步骤，避免广播唤醒后大量线程争抢同一互斥锁又重新睡眠。F7LY 在 `futex` 等待队列层完成迁移，保持条件变量和互斥锁交接的一致性。

7.3 管道与 FIFO

管道是以内核缓冲区中转的字节流。F7LY 将管道端点实现为 `VFS` 文件对象，因此匿名管道、命名管道和普通 `fd` 读写都复用同一套打开、关闭、阻塞和非阻塞语义。

7.3.1 管道语义

一个管道由共享循环缓冲区和两类端点组成：读端从缓冲区取出字节，写端向缓冲区追加字节。缓冲区状态、读写端是否仍打开、等待读者和等待写者数量共同决定读写行为。

写入时，如果缓冲区仍有空间，数据进入缓冲区并唤醒读端等待者；如果缓冲区已满，非阻塞写返回重试错误，阻塞写则睡眠等待空间。读出时，如果缓冲区有数据，读端复制并唤醒写端；如果没有数据，写端仍打开时读端阻塞，写端全部关闭时读端返回 `EOF`。读端全部关闭后，写端收到管道破裂语义并返回错误。

7.3.2 FIFO 管理器

匿名管道依靠继承 fd 传递端点，命名管道则以路径作为共享身份。F7LY 的 FIFO 管理器维护“路径到管道对象”的映射，并记录当前读者和写者数量。首次打开某个 FIFO 路径时创建底层管道，后续打开同一路径复用该管道；当读写两端引用都归零时，管理器释映射和缓冲区。

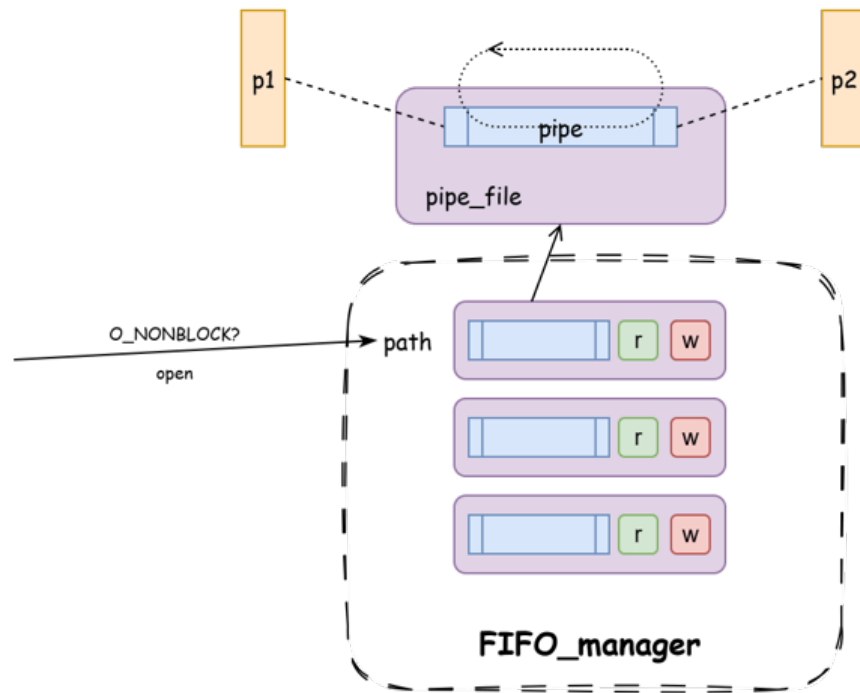


图 7-2 fifomanager

打开 FIFO 时，路径映射和端点计数共同决定打开结果。当前实现重点覆盖非阻塞只写且没有读者时返回 `ENXIO`、`O_RDWR` 同时保留读写端、以及后续读写阶段的阻塞/非阻塞语义。路径映射和引用计数由锁保护，避免并发打开/关闭破坏同一个 FIFO 的共享身份。

7.4 共享内存与 memfd

共享内存允许多个进程把同一批物理页映射到各自地址空间，数据读写不再经过内核中转。F7LY 同时支持 SysV 共享内存和 memfd：前者以全局 key 查找共享段，后者以文件描述符传递共享对象。

7.4.1 SysV 共享内存

SysV 共享内存的思路是：内核维护一组命名的共享内存段，进程通过一个整数 `key` 找到目标段，将它映射到自己的地址空间，之后直接通过指针读写，数据全程走物理内存，内核不参与搬运。

```
struct shm_segment
{
    int shmidx;
    key_t key;
    size_t size;
    size_t real_size;
    u16 shmflg;
    eastl::vector<attached_entry> attached_addrs;
    SysvShmVmObject *object;
    int nattch;
    bool auto_destroy_on_last_detach;
    uid_t owner_uid;
    gid_t owner_gid;
    pid_t creator_pid;
    time_t atime, dtime, ctime;
    // .....
};
```

进程调用 `shmget`，传入一个 `key`，不同进程用同一个 `key` 就能找到同一个段。如果该 `key` 的段已经存在，直接返回它的 ID；如果不存在，需要在 `flags` 里带上 `IPC_CREAT` 才会现场创建一个。

每个段记录了自己的大小、访问权限（谁可读、谁可写），以及当前有哪些进程把它映射到了哪个虚拟地址。物理页不直接挂在段上，而是由一个专门的 `SysvShmVmObject` 对象管理，页面的分配和回收都走这个对象。这套 `VmObject` 体系也同时服务于 `mmap MAP_SHARED` 的文件映射，两种共享方式在物理页管理层面上是统一的。

`shmat` 将段附加到进程地址空间，返回一个可直接访问的虚拟地址；`shmdt` 解除附加并减少引用。`shmctl IPC_RMID` 只设置待删除标记：若仍有附加者，段会保留到 `nattch` 归零后销毁；若没有附加者，则立即销毁。没有打删除标记的普通 SysV 段即使暂时无人附加也继续存在；`auto_destroy_on_last_detach` 只用于内核借用

同一后端对象的自动清理场景。`fork` 会继承已有附加关系，进程退出时也会自动解除自己持有的附加。

7.4.2 memfd

`memfd` 是基于 `fd` 的匿名共享内存。创建方得到一个匿名文件 `fd` 后，通过共享映射写入数据，也可以经 `Unix domain socket` 把 `fd` 传给其他进程。生命周期由 `fd` 引用和映射引用自然管理，不需要全局 `key`。

F7LY 的 `memfd` 借助 VFS 临时文件能力承载数据，同时为 `memfd` 维护一份共享状态：文件大小、密封位和是否允许继续添加密封。密封一旦设置不可撤销，并在所有修改路径上强制生效。例如写密封会拒绝新的可写共享映射和直接写入，缩小/增长密封会限制截断和预分配。这样 `memfd` 可以作为安全的只读快照或受限共享缓冲区使用。

7.5 就绪通知

多路复用与事件通知把“多个 `fd` 的状态变化”统一成可等待事件。F7LY 实现了 `epoll` 和 `eventfd`，并让它们都接入 VFS 文件对象体系。

7.5.1 epoll

`epoll` 将“注册关注 `fd`”和“等待就绪”分离。关注列表变化时通过控制接口更新，等待时内核只检查已注册的条目，避免每次从用户态重新传入完整 `fd` 集合。每个关注条目保存目标 `fd`、事件掩码、用户数据以及边沿触发和 `oneshot` 所需的标志。

`epoll` 不为每种 `fd` 单独写轮询逻辑，而是调用文件对象的读写就绪判断。管道、`socket`、设备和 `eventfd` 各自定义什么叫“可读/可写”，`epoll` 层只负责事件掩码匹配和批量返回。`epoll fd` 可以被另一个 `epoll` 关注，用于嵌套关系和环路检查；当前 `epoll` 文件自身不通过普通 `read_ready / write_ready` 产生读写就绪事件。

水平触发模式下，只要 `fd` 保持就绪就会持续通知；边沿触发模式只在状态从未就绪变为就绪时通知一次；`oneshot` 条目触发后自动禁用，等待用户重新激活。`epoll` 嵌套时内核限制嵌套深度并检测环路，防止自引用导致无限遍历。

7.5.2 eventfd

`eventfd` 是一个以 `fd` 形式暴露的 64 位事件计数器。写入会累加计数，读取会取走当前值并清零；信号量模式下，每次读取只消耗 1。由于 `eventfd` 同样实现读写就绪判断，它可以被 `epoll` 监控，成为事件循环中的可编程唤醒源。写入后，等待循环下一次检查会看到该 `fd` 可读并返回事件；当前实现没有为 `eventfd` 单独维护 `wait queue`，而是复用 `fd` 事件等待轮询。

第八章 系统调用

8.1 系统调用概述

系统调用是用户程序进入内核服务的统一入口。F7LY 实现了与 Linux raw syscall 约定兼容的接口，覆盖进程管理、内存管理、文件系统、信号、时间、futex、事件通知、网络、权限和系统信息等功能类别。完整实现列表放在附录，本章只说明系统调用层如何分发请求、搬运参数并连接内核子系统。

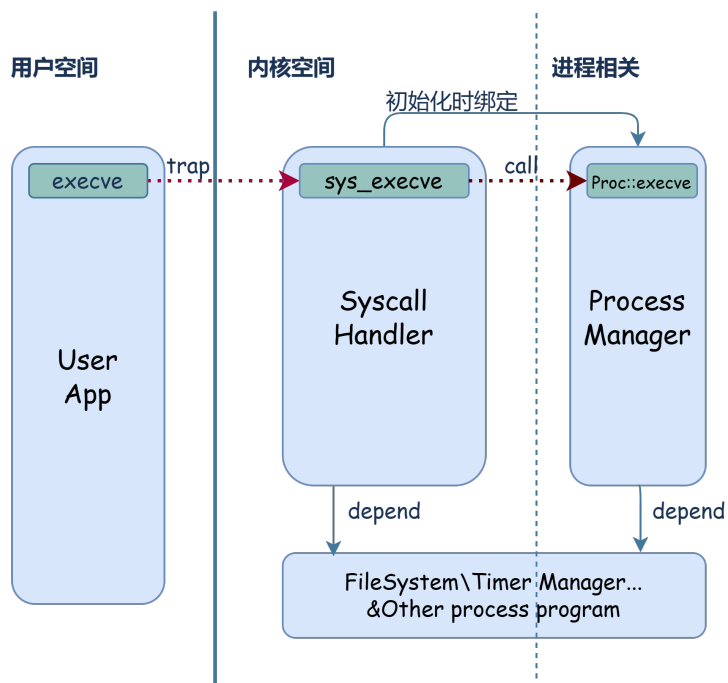


图 8-1 系统调用执行流程

用户态程序把系统调用号和参数放入约定寄存器后执行陷入指令。架构 trap 入口保存现场并识别这是系统调用，再交给统一分发器处理。分发器根据调用号找到处理逻辑，执行完成后把返回值写回用户态返回寄存器。内核侧统一返回 Linux 风格错误码；表内未实现调用会落到默认处理并返回 `-ENOSYS`，调用号越界或表项异常时当前分发器兜底返回 `-1`。

系统调用的管理由 `SyscallHandler` 类完成，在内核启动的 `main()` 函数中统一完成系统调用表的初始化绑定。其核心结构示例如下：


```

class SyscallHandler
{
    // 使用了这个类的成员函数指针
    using SyscallFunc = uint64 (SyscallHandler::*)(); // 定义一个函数指针类型 SyscallFunc
private:
    SyscallFunc _syscall_funcs[max_syscall_funcs_num]; // 存储系统调用函数的指针数组
    const char *_syscall_name[max_syscall_funcs_num]; // 存储系统调用名称的指针数组
    uint64_t _default_syscall_impl(); // 默认的系统调用实现
public:
    void init(); // 使用构造函数进行init
    void invoke_syscaller(); // 调用系统调用
    SyscallHandler()
    {
    }

private:
    int _fetch_addr(uint64 addr, uint64 &out_data);
    int _fetch_str(uint64 addr, eastl::string &buf, uint64 max);
    uint64 _arg_raw(int arg_n);

    int _arg_int(int arg_n, int &out_int);
    int _arg_long(int arg_n, long &out_int);
    int _arg_addr(int arg_n, uint64 &out_addr);
    int _arg_str(int arg_n, eastl::string &buf, int max);
    int _arg_fd(int arg_n, int *out_fd, fs::file **out_f);
    int argfd(int n, int *pfd, struct file **pf);
    bool is_bad_addr(uint64 addr);

};

extern SyscallHandler k_syscall_handler;

```

8.2 执行流程

系统调用的一次执行可以分为四步：

1. **用户态陷入**：用户程序按照 ABI 放置调用号和最多六个参数，执行陷入指令进入内核。
2. **trap 分发**：架构入口判断异常类型，推进返回地址，打开允许的中断窗口，并转入系统调用分发器。
3. **查表执行**：分发器检查调用号范围和绑定状态，调用对应处理逻辑；处理逻辑只通过安全参数接口访问用户态地址和 fd。
4. **返回用户态**：返回值写回陷阱帧，随后内核处理待投递信号、调度抢占和用户态现场恢复。

这个流程把架构差异限制在 trap 入口和寄存器约定中，系统调用实现本身尽量复用同一套 C++ 逻辑。

8.3 分发表设计

F7LY 使用固定容量的系统调用表作为分发核心。初始化时，表中所有槽位先指向默认未实现处理；随后按系统调用号绑定已实现入口。调用名称也随绑定记录下来，用于诊断输出和日志定位。

这种设计有三个好处。第一，表内未实现调用有统一兜底，不会落入空指针。第二，新增系统调用只需要同步编号、声明、实现和绑定位置，入口路径保持一致。第三，RISC-V 与 LoongArch 共用同一张逻辑表，保证两架构看到的 Linux ABI 编号一致。

分发器本身只做边界检查、入口调用、返回值写回和少量一致性检查。具体语义放在各系统调用处理逻辑或内核子系统中，避免把 VFS、进程、内存、网络等细节堆在分发层。

8.4 参数与用户内存

系统调用参数先按原始寄存器值读取，再根据目标类型进行解释。整数参数按目标类型提取，地址参数只作为用户虚拟地址保存，字符串和结构体必须通过页表感知的拷贝函数搬入内核，输出数据则通过对称的拷出函数写回用户态。内核不会直接解引用用户指针。

文件描述符参数需要额外经过 fd 表校验。内核先检查 fd 编号范围，再取得当前进程的文件对象，并确认对象仍然有效。这样系统调用处理逻辑拿到的是已经验证过的 file 抽象，可以直接转交 VFS、socket、pipe、eventfd 或设备后端。

这一层的核心原则是“先校验边界，再进入子系统”。无效地址返回 `-EFAULT`，非法 fd 返回 `-EBADF`，参数不符合语义返回对应 Linux `errno`。错误码在系统调用层保持负值，用户态 C 库再按约定转换成 `errno`。

8.5 实现分层

系统调用实现不是按文件简单堆叠，而是围绕几类职责拆分：

- **ABI 数据结构**：定义用户态可见结构体的布局、对齐和大小，保证 RISC-V 与 LoongArch 下 `copy_in / copy_out` 解释一致。
- **进程与凭据**：处理 pid/tid、会话、进程组、uid/gid、capability、资源限制和 personality 等进程属性。
- **内存接口**：把 `brk`、`mmap`、`munmap`、`mprotect`、共享内存和 `memfd` 请求转交地址空间与页后端对象。
- **文件与 I/O**：把 `open/read/write/stat/mount/ioctl/fcntl/xattr` 等请求转交 VFS、文件对象和设备控制层。
- **信号、时间与同步**：连接信号处理表、POSIX timer、futex、epoll、eventfd 和等待唤醒机制。
- **网络与设备控制**：socket 系列系统调用进入网络子系统，块设备、终端和 socket 的 `ioctl` 按设备类型分发。

系统调用处理函数在这一分层中扮演“ABI 边界适配器”的角色：它负责把用户态参数转换成内核对象和安全缓冲区，再调用真正持有状态的子系统。这样修改某个子系统的内部实现时，不需要改变所有系统调用入口。

8.6 新增系统调用的约束

新增系统调用时需要同步四类信息：Linux ABI 编号、内核侧声明、处理逻辑和分发表绑定。若用户态封装或自动入口需要调用它，还要同步用户态 wrapper。实现时应优先复用已有的参数获取、fd 校验、用户拷贝和 `errno` 风格，避免每个系统调用各自实现一套边界检查。

对于已经有子系统承载的能力，系统调用层只做入口适配，不在分发文件里重写业务逻辑。例如文件路径解析交给 VFS，页映射交给地址空间管理，事件等待交给 `epoll/file` 就绪判断，设备命令交给对应 `ioctl` 模块。这是保持系统调用层可维护的主要原则。

第九章 网络系统模块

9.1 网络系统架构概述

网络系统是用用户程序与外部主机进行通信的重要基础设施，允许用户程序通过 `Socket` 接口完成数据发送、接收、连接建立、端口监听、事件等待等操作。F7LY 实现了与 Linux ABI 兼容的网络系统调用接口，并在 2026 年进一步完善本机 TCP/UDP 数据面能力，使网络子系统从早期的协议栈框架与 `Socket` 接口占位，推进到能够真实传递 `payload` 的可用阶段。目前，F7LY 已支持 `send / recv`、`sendto / recvfrom`、`poll / epoll` 等关键接口，能够支撑 `iperf`、`netperf`、`BusyBox` 网络工具以及部分 LTP 网络用例完成真实的数据传输与就绪通知验证。

当前网络模块分为三层：

- 第一层是 Linux 兼容的 BSD `Socket` 接口。系统调用层负责参数获取、用户态地址拷贝、`fd` 分配和错误码转换，真正的 `socket` 状态机集中在 `socket_file` 中。
- 第二层是内核内 `loopback` 数据面。`127.0.0.1`、`0.0.0.0` 以及可映射的 `IPv6 loopback` 地址不经过网卡，也不经过完整 `Ethernet/IP` 层，而是在内核内通过端口表、`peer` 指针和接收队列传递数据。
- 第三层是 `ONPS` 与 `VirtIO-Net` 适配后端。非 `loopback` 的 `IPv4 TCP/UDP/ICMP` 流量在网络栈初始化成功后可以交给 `ONPS`，`ONPS` 再通过 `virtio0` 适配层收发完整以太网帧。

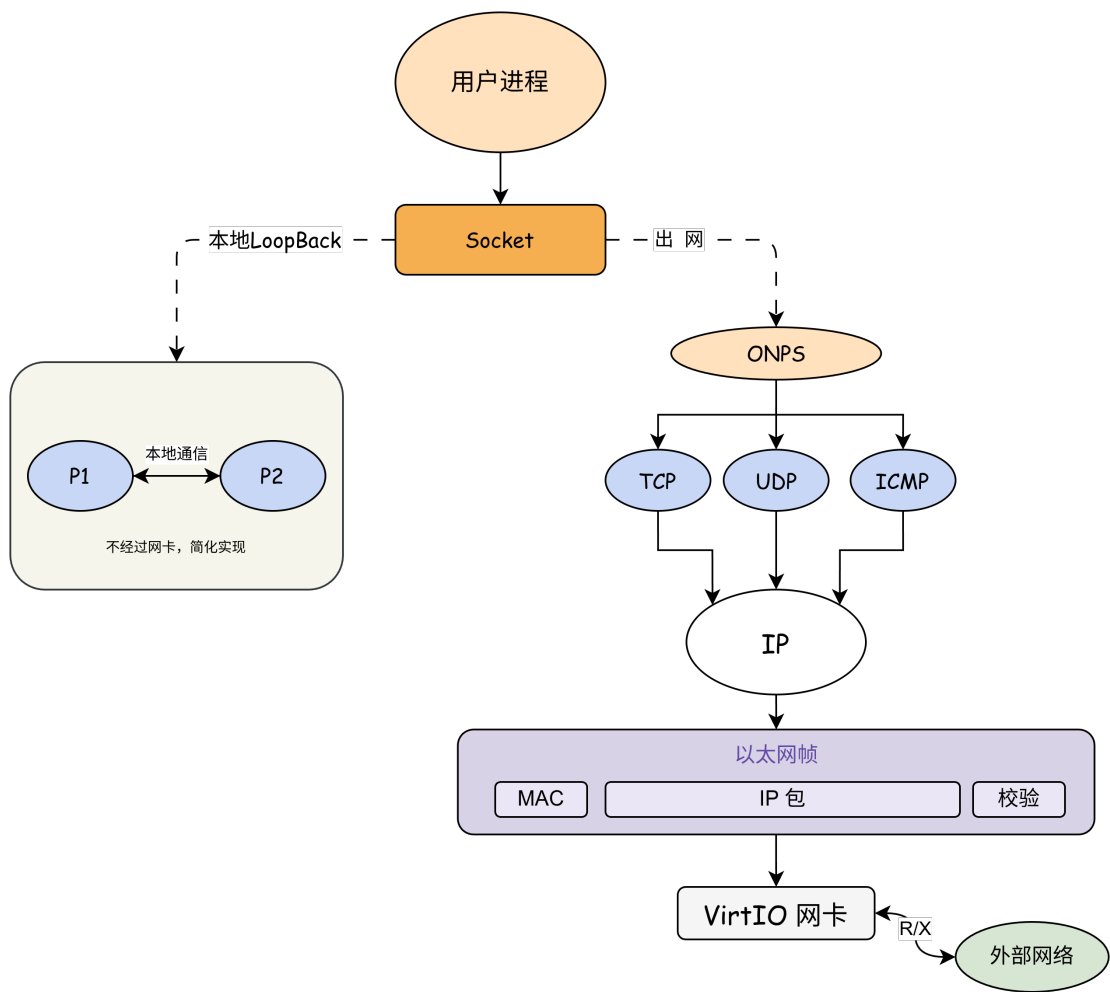


图 9-1 网络模块架构示意图

图 15 展示了这一分层关系。需要特别说明的是，F7LY 采用本机 loopback 与 ONPS/VirtIO-Net 双路径框架，是基于实现边界清晰和功能验证稳定性的考虑。这样的划分避免了两个问题：一方面，localhost 通信不必受真实网卡、QEMU user 网络、ARP 或外部路由状态影响；另一方面，外部网络仍然保留了从协议栈到 VirtIO 设备的完整接入点，后续可以继续扩展。

网络栈初始化采用懒加载方式。第一次创建 IPv4 TCP/UDP/ICMP socket 时，`sys_socket()` 会尝试调用 `net::init_network_stack()`。

```
bool init_network_stack()
{
    if (network_initialized) {
        return true;
    }
}
```

```

    }

    EN_ONPSERR onps_error;
    if (!open_npstack_load(&onps_error)) {
        return false;
    }

    if (!net::adapter_init()) {
        open_npstack_unload();
        return false;
    }

    network_initialized = true;
    return true;
}

```

这段初始化顺序体现了网络模块的两个边界。`open_npstack_load()` 只负责启动 ONPS 协议栈核心, 包括 `buddy`、`buf_list`、定时器、输入线程、网络接口表和路由表等基础结构; `adapter_init()` 才负责把底层 VirtIO-Net 设备注册成 ONPS 可见的以太网接口。`Socket` 层在转发前通过 `should_route_via_onps()` 判断目标地址是否为 `loopback`, 从而决定走内核内路径还是 ONPS 后端路径。

9.2 核心网络协议栈与 loopback 数据面

本节重点分析 F7LY 的 `loopback` 实现。当前 `loopback` 实现方式为内核内部的 `socket_file` 层直接处理本机通信, 绕过 VirtIO 网卡和完整的 Ethernet/IP 层。这种实现是基于性能和实现复杂度的考虑。实现时保留了 ONPS 的 `loopback` 模块, 以确保后续能够无缝切换到 ONPS 后端。

实现思路为: 本机通信不经过 VirtIO 网卡, 也不经过 Ethernet/IP 层。用户态看到的仍然是标准 BSD Socket 接口, 但内核内部会把 `loopback` 流量留在 `socket_file` 层, 通过端口表、`peer` 指针、接收缓冲区和等待队列完成传输。

整体路径可以概括为:

```

TCP loopback:
socket/connect -> 查找 listener -> 创建 server-side socket

```

```

-> client/server-side 互设 peer
-> send 写入 peer->_recv_buffer
-> recv 从本端 _recv_buffer 读取

```

UDP loopback:

```

socket/bind -> 注册 UDP 端口
sendto      -> 按目标端口查找 receiver
            -> 入队一枚 loopback_datagram
recvfrom    -> 出队 datagram 并回填源地址

```

这一实现和真实网卡路径是分开的。`socket_file` 先判断目标地址是否属于本机通信；只有目标不是 loopback/any，且网络栈已经初始化成功时，才把流量转交给 ONPS。

```

bool is_loopback_or_any(uint32 addr)
{
    return addr == 0 || addr == k_loopback_addr || addr == 0x7f000001;
}

bool should_route_via_onps(SocketFamily family, SocketType type,
uint32 addr)
{
    return can_use_onps_socket(family, type) && !
is_loopback_or_any(addr);
}

```

9.2.1 loopback 端口表

loopback 数据面的入口是本机端口表。F7LY 用 `g_loopback_bindings` 记录已经绑定的 TCP/UDP 端口，表项由 socket 类型和端口号共同确定。TCP listener 和 UDP receiver 都通过这张表被找到。没有显式 `bind()` 的 socket 会在 `connect()`、`listen()` 或首次 `sendto()` 前自动分配临时端口，范围是 `20000..60999`。

```

constexpr int k_loopback_binding_max = 256;
constexpr uint16 k_ephemeral_port_start = 20000;
constexpr uint16 k_ephemeral_port_end = 60999;

```



```

struct loopback_binding
{
    bool used = false;
    uint16 port = 0;
    SocketType type = SocketType::TCP;
    socket_file *socket = nullptr;
};

SpinLock g_loopback_lock;
loopback_binding g_loopback_bindings[k_loopback_binding_max];

```

端口表解决的是“本机路由”问题：TCP `connect()` 用目标端口找到正在 `listen()` 的 socket；UDP `sendto()` 用目标端口找到接收端 socket；`accept()` 则从 listener 自己的 pending 队列中取出已经建立好的 server-side socket。这样，loopback 后端不需要构造 IP 包，也不需要经过 ARP、路由表和网卡队列。

9.2.2 TCP loopback 方法

TCP loopback 的核心是成对 `socket_file`。客户端调用 `connect()` 时，内核按目标端口查找 listener。如果服务端线程还在启动过程中，`connect()` 会短暂等待 listener 完成 `bind/listen`，以降低 `netperf` 这类程序中 server/client 启动竞态的影响。

找到 listener 后，内核创建一个新的 server-side `socket_file`，将它与 client socket 互相挂为 `_peer`，再把 server-side 放入 listener 的 `_pending_connections` 队列。服务端 `accept()` 取出的不是原 listener，而是这个已经与客户端互连的新 socket。

```

class socket_file : public file
{
private:
    eastl::vector<socket_file*> _pending_connections;
    int _backlog;
    socket_file *_peer;

    eastl::vector<uint8_t> _recv_buffer;

```

```

    eastl::vector<uint8_t> _send_buffer;
    bool _read_shutdown;
    bool _write_shutdown;
    bool _peer_closed;
    bool _peer_write_shutdown;
};

```

连接建立后, TCP 数据发送不再创建网络包, 而是直接写入对端 socket 的接收缓冲区。TCP 是字节流协议, 所以 `_recv_buffer` 不保存单次 `send()` 的边界; 接收方每次 `recv()` 只是从字节流头部取走指定长度的数据。

为了避免单个连接无限占用内核内存, TCP loopback 给每个接收缓冲区设置了 512KB 上限。发送方发现对端缓冲区满时, 阻塞 socket 会睡眠等待; 非阻塞 socket 或带 `MSG_DONTWAIT` 的发送会返回 `-EAGAIN`。接收方读出数据后唤醒写端, 形成一个明确的背压闭环。

```

constexpr size_t k_tcp_recv_buffer_max_bytes = 512 * 1024;

int socket_file::enqueue_stream_data_to_peer(socket_file *peer,
                                              const uint8_t *data,
                                              size_t len,
                                              bool nonblocking)
{
    size_t queued = 0;
    while (queued < len) {
        peer->_lock.acquire();
        while (peer->_recv_buffer.size() >=
k_tcp_recv_buffer_max_bytes &&
            peer->stream_receive_open_locked()) {
            if (nonblocking) {
                peer->_lock.release();
                return queued > 0 ? static_cast<int>(queued) : -
EAGAIN;
            }
            proc::k_pm.sleep(&peer->_recv_buffer, &peer->_lock);
        }
    }
}

```

```

size_t used = peer->_recv_buffer.size();
size_t space = used < k_tcp_recv_buffer_max_bytes
                ? k_tcp_recv_buffer_max_bytes - used
                : 0;

size_t chunk = eastl::min(len - queued, space);
size_t old_size = peer->_recv_buffer.size();
peer->_recv_buffer.resize(old_size + chunk);
memcpy(peer->_recv_buffer.data() + old_size, data + queued,
chunk);
queued += chunk;

proc::k_pm.wakeup(&peer->_recv_buffer);
peer->_lock.release();
}
return static_cast<int>(queued);
}

```

在基本字节流之外，TCP loopback 还实现了用户态程序常见的边界语义：

- MSG_MORE 会先把数据暂存在 `_send_buffer`，等下一次非 MSG_MORE 发送时再合并写入对端；
- shutdown(SHUT_RD/SHUT_WR) 会分别关闭读半边或写半边，并唤醒对端，使 `recv()` 能正确看到 EOF 或错误；
- SO_RCVTIMEO 参与阻塞接收的 deadline 计算，等待超时后返回对应错误；
- 等待期间收到未屏蔽信号时，阻塞系统调用返回 `-EINTR`；
- `read_ready()`、`write_ready()` 和 `epoll_rdhup_ready()` 对接 `poll/select/epoll`。

因此，F7LY 的 TCP loopback 不只是把两个 socket 粘在一起，而是实现了连接队列、流式缓冲、背压、半关闭、超时、信号中断和就绪通知这些真实程序会依赖的行为。

9.2.3 UDP loopback 方法

UDP loopback 与 TCP 的最大区别是必须保留报文边界。一次 `sendto()` 对应接收端队列中的一枚 datagram，`recvfrom()` 一次只取一枚，并把源地址回填给用户态。

```

struct loopback_datagram
{
    struct sockaddr_in src_addr;
    eastl::vector<uint8_t> data;
};

eastl::vector<loopback_datagram> _datagram_queue;
size_t _datagram_queue_bytes;

```

发送时，`socket_file` 先确定目标地址。如果目标是非 loopback IPv4，并且 ONPS 网络栈可用，则走出网路径；否则按目标端口在 `g_loopback_bindings` 中查找 UDP receiver。找到接收者后，内核把源地址和 payload 打包成一枚 `loopback_datagram`，压入接收端 `_datagram_queue` 并唤醒阻塞的 `recvfrom()`。

UDP 队列同样有上限：最多 256 个报文，或最多 256KB payload。队列满时当前实现会丢弃新到达的 datagram，但 `sendto()` 仍按 UDP 语义返回成功。这一点很重要：UDP 本身不是可靠协议，loopback 后端不能因为在内核内实现就把它改造成可靠队列。

```

constexpr size_t k_udp_queue_max_bytes = 256 * 1024;
constexpr size_t k_udp_queue_max_packets = 256;

int socket_file::enqueue_datagram(const struct sockaddr_in *src_addr,
                                   const uint8_t *data,
                                   size_t len)
{
    if (_datagram_queue.size() >= k_udp_queue_max_packets ||
        _datagram_queue_bytes + len > k_udp_queue_max_bytes) {
        return static_cast<int>(len);
    }

    loopback_datagram packet;
    if (src_addr != nullptr) {
        packet.src_addr = *src_addr;
    }
    packet.data.resize(len);
    if (len > 0) {

```

```

        memcpy(packet.data.data(), data, len);
    }
    _datagram_queue_bytes += len;
    _datagram_queue.push_back(packet);
    proc::k_pm.wakeup(&_datagram_queue);
    return static_cast<int>(len);
}

```

UDP 还支持 connected UDP 语义。调用 `connect()` 后，socket 会记录 remote 地址，后续 `send()` 可以复用这个目标；接收侧仍然保持 datagram 队列和源地址回填。若目标端口没有接收者，`sendto()` 按 UDP 语义允许成功返回，而不是强制要求连接存在。

9.2.4 IPv6 loopback 兼容

F7LY 目前没有完整 IPv6 协议栈，但许多用户态程序会优先创建 `AF_INET6` socket 做双栈监听。为兼容这类程序，`socket_file` 只支持把本机相关 IPv6 地址映射到 IPv4 loopback：

- `::` 映射为 `0.0.0.0`；
- `:::1` 映射为 `127.0.0.1`；
- IPv4-mapped IPv6 地址映射为其尾部 IPv4 地址。

不可映射的 IPv6 地址直接返回不支持。这个边界比较清晰：当前实现不是完整 IPv6 网络，而是让双栈 localhost 程序复用已经实现的 IPv4 loopback 数据面。

9.3 ONPS 与 VirtIO 出网路径

本节说明非 loopback IPv4 流量如何走出内核内数据面。F7LY 在出网路径上复用 ONPS 作为 TCP/IP 协议栈后端，再通过 VirtIO-Net 适配层把 ONPS 的以太网接口接到 QEMU 暴露的虚拟网卡上。换言之，loopback 是 F7LY 自己实现的本机数据面；出网路径则是 `socket_file -> ONPS -> virtio0 -> VirtIO-Net`。

VirtIO-Net 驱动承担的是“收发完整以太网帧”的职责。它不理解 TCP 连接、UDP 报文或 socket 状态，也不直接接触用户态 fd；上层 ONPS 给它一个已经组装好的 Ethernet frame，它把帧放入 TX virtqueue 交给设备；设备收到包后写入 RX virtqueue，它取出帧并交回 ONPS。

F7LY 在双架构下使用不同的 VirtIO 传输方式：

- RISC-V 平台扫描 `VIRTIO_MMIO_FIRST` 到 `VIRTIO_MMIO_LAST` 之间的 MMIO 槽位，查找 `device_id = 1` 的网络设备；
- LoongArch 平台通过 PCI 探测 `VIRTIO_NET_VENDOR_ID` 和 `VIRTIO_NET_DEVICE_ID`，再配置 modern PCI virtqueue；
- 两条路径最终都初始化同一份 `virtio_net_device` 状态，包括 RX/TX 队列、描述符数组、available ring、used ring、MAC 地址和设备锁。

驱动当前只协商 `VIRTIO_NET_F_MAC`。校验和卸载、TSO、mergeable buffer 等复杂特性没有启用，原因是 F7LY 的网络目标不是在这一层做性能优化，而是先保证协议栈能够稳定收发完整帧，并让上层对数据边界和错误语义有确定行为。

```
uint64 negotiate_features(uint64 device_features)
{
    // 当前驱动只收发完整以太网帧，不启用校验和、TSO、mergeable buffer。
    return device_features & (1ULL << VIRTIO_NET_F_MAC);
}
```

9.3.1 RX/TX 队列组织

VirtIO-Net 使用两个 virtqueue：RX 队列由设备写入，TX 队列由驱动写入。每个队列当前使用 `NUM_NET_DESC = 32` 个描述符。初始化时驱动会一次性向 RX 队列投递全部可写 buffer，否则设备即使收到网络包也没有 DMA 目标。

发送路径的关键步骤是：

1. 主动回收已经完成的 TX used ring，释放旧 descriptor；
2. 找到空闲 TX descriptor；
3. 在包前写入清零的 `virtio_net_hdr`；
4. 将完整以太网帧复制到 header 后方；
5. 把 descriptor index 放入 avail ring；
6. notify TX queue。

```
int virtio_net_send(const void *data, uint32 len)
{
    process_tx_used_locked();
}
```

```

    virtio_net_hdr *hdr =
        reinterpret_cast<virtio_net_hdr
*>(g_net.tx_buffers[buf_idx].data);
    memset(hdr, 0, sizeof(*hdr));
    memcpy(g_net.tx_buffers[buf_idx].data + k_net_header_len, data,
len);

    g_net.tx_desc[desc_idx].addr =
dma_addr(g_net.tx_buffers[buf_idx].data);
    g_net.tx_desc[desc_idx].len = len + k_net_header_len;
    g_net.tx_desc[desc_idx].flags = 0;

    g_net.tx_avail[2 + (idx % NUM_NET_DESC)] = desc_idx;
    __sync_synchronize();
    set_avail_idx(g_net.tx_avail, idx + 1);
    notify_queue(VIRTIO_NET_TX_QUEUE_IDX);
    return 0;
}

```

接收路径则从 RX used ring 中取出设备已经写好的 descriptor，去掉前置的 virtio_net_hdr，把后面的以太网帧复制给上层，然后立即把同一个 descriptor 重新投递回 RX 队列。这样 RX buffer 始终处于循环复用状态。

```

int virtio_net_recv(void *data, uint32 *len)
{
    if (g_net.rx_used_idx == g_net.rx_used->idx) {
        return -1;
    }

    VRingUsedElem *elem = &g_net.rx_used->ring[g_net.rx_used_idx %
NUM_NET_DESC];
    uint32 desc_idx = elem->id;
    uint32 used_len = elem->len;
    ++g_net.rx_used_idx;

    uint32 data_len = used_len - k_net_header_len;
    memcpy(data, g_net.rx_buffers[buf_idx].data + k_net_header_len,

```

```

copy_len);

    g_net.rx_free[desc_idx] = 1;
    post_rx_desc_locked(desc_idx);
    notify_queue(VIRTIO_NET_RX_QUEUE_IDX);
    return 0;
}

```

这一路径保持了驱动层的职责单一：只处理 DMA、virtqueue、MAC 和帧边界，不把 socket 语义泄漏到底层设备中。

9.3.2 ONPS 适配层

ONPS 原本需要一个以太网设备抽象，而 F7LY 的 VirtIO-Net 驱动暴露的是 `virtio_net_send()` 和 `virtio_net_recv()`。两者之间由 `virtio_net_adapter.cc` 衔接。适配层在初始化时读取 VirtIO MAC 地址，构造 QEMU user-mode 网络常用配置，并通过 `ethernet_add()` 注册接口：

```

ST_IPV4 ipv4_config;
ipv4_config.unAddr = inet_addr("10.0.2.15");
ipv4_config.unSubnetMask = inet_addr("255.255.255.0");
ipv4_config.unGateway = inet_addr("10.0.2.2");
ipv4_config.unPrimaryDNS = inet_addr("10.0.2.3");
ipv4_config.unBroadcast = inet_addr("10.0.2.255");

onps_netif = ethernet_add("virtio0",
                           mac_addr,
                           &ipv4_config,
                           virtio_emac_send,
                           start_recv_thread_wrapper,
                           &onps_netif,
                           &error);

```

发送方向上，ONPS 给出的不是连续内存，而是内部 `buf_list` 链表。`virtio_emac_send()` 先计算总长度，再将链表合并到一块连续的 `tx_packet_buffer`，最后调用 `virtio_net_send()`。接收方向上，适配层创建一个名为 `virtio-net-rx` 的内核线程，循环调用 `virtio_net_recv()`。一旦取到完整以

以太网帧，就通过 `ethernet_ii_recv(netif, rx_packet_buffer, packet_len)` 交还给 ONPS 解析。

```
int virtio_emac_send(short buf_list_head, unsigned char *error)
{
    UINT total_len = buf_list_get_len(buf_list_head);
    if (total_len <= 0 || total_len > ETH_FRAME_LEN) {
        return -1;
    }

    buf_list_merge_packet(buf_list_head, tx_packet_buffer);
    return net::virtio_net_send(tx_packet_buffer, total_len) == 0
        ? static_cast<int>(total_len)
        : -1;
}
```

适配层的价值在于隔离两种模型：ONPS 仍然按照自己的网卡回调、`buf_list` 和 `ethernet_ii_recv()` 工作；F7LY 驱动层仍然按照 VirtIO queue 和连续 DMA buffer 工作。二者之间没有互相侵入，也没有把 ONPS 的内部结构扩散到设备驱动中。

9.4 BSD Socket 接口与系统调用集成

网络模块对用户态暴露的是 Linux 风格 BSD Socket ABI。F7LY 没有为 socket 建立一套独立于文件系统的对象体系，而是让 `socket_file` 继承 VFS 的 `file` 基类。这一点非常关键：socket 一旦成为普通 fd，就可以自然参与 `read/write`、`close`、`dup`、`fcntl`、`poll`、`select`、`epoll`、`fork` 后 fd 继承等通用路径。

```
class socket_file : public file
{
public:
    virtual long read(uint64 buf, size_t len, long off, bool upgrade)
    override;
    virtual long write(uint64 buf, size_t len, long off, bool upgrade)
    override;
    virtual bool read_ready() override;
    virtual bool write_ready() override;
```

```

int bind(const struct sockaddr *addr, socklen_t addrlen);
int listen(int backlog);
int accept(struct sockaddr *addr, socklen_t *addrlen,
           socket_file **accepted_socket);
int connect(const struct sockaddr *addr, socklen_t addrlen);
int sendmsg(const struct msghdr *msg, int flags);
int recvmsg(struct msghdr *msg, int flags);
};

```

在 VFS 视角下，socket 和 pipe、普通文件、设备文件一样都是 `file`。区别只在于各自的后端实现不同：普通文件读写 ext4/FAT32，管道读写环形缓冲区，socket 则根据地址和协议族选择 loopback、AF_UNIX 或 ONPS 后端。

9.4.1 系统调用层职责

系统调用层主要承担 ABI 边界工作，而不直接实现协议状态机。以 `socket()` 为例，`sys_socket()` 会拆出 `SOCK_CLOEXEC` 和 `SOCK_NONBLOCK`，校验协议族、类型和协议号，然后创建 `socket_file` 并放入当前进程 fd 表。支持的协议族包括 `AF_UNIX`、`AF_INET` 和用于 loopback 兼容的 `AF_INET6`；支持的类型包括 `SOCK_STREAM`、`SOCK_DGRAM`、`SOCK_RAW`，其中 `AF_UNIX` 的 `SOCK_SEQPACKET` 在内部复用可靠本地 stream 语义。

```

uint64 SyscallHandler::sys_socket()
{
    int domain, type, protocol;
    if (_arg_int(0, domain) < 0 ||
        _arg_int(1, type) < 0 ||
        _arg_int(2, protocol) < 0) {
        return SYS_EINVAL;
    }

    int fd_flags = type & (O_CLOEXEC | O_NONBLOCK);
    int base_type = type & ~(O_CLOEXEC | O_NONBLOCK);

    if (domain == AF_INET &&
        (base_type == SOCK_STREAM || base_type == SOCK_DGRAM ||

```

```

base_type == SOCK_RAW) &&
    !net::is_network_stack_ready()) {
    net::init_network_stack();
}

fs::socket_file *sock = new fs::socket_file(domain, base_type,
protocol);
sock->lwext4_file_struct.flags = O_RDWR | fd_flags;
sock->set_nonblock((fd_flags & O_NONBLOCK) != 0);

proc::Pcb *p = proc::k_pm.get_cur_pcb();
int fd = proc::k_pm.alloc_fd(p, sock);
if (fd >= 0 && (fd_flags & O_CLOEXEC) != 0) {
    p->_ofile->_fl_cloexec[fd] = true;
}
return fd;
}

```

`bind()`、`listen()`、`connect()`、`accept()`、`sendto()`、`recvfrom()` 等调用同样遵循这个分工。`syscall handler` 负责 `fd` 类型检查、`copy_in / copy_out`、`sockaddr` 长度校验和返回值包装；地址归一化、端口登记、`listener` 查找、队列入队、阻塞等待和错误语义由 `socket_file` 统一处理。这样可以避免 `syscall` 层和 `socket` 后端同时修改状态，降低竞态和语义分叉。

当前已绑定的主要网络系统调用包括：

- 连接管理：`socket`、`socketpair`、`bind`、`listen`、`accept`、`accept4`、`connect`、`shutdown_socket`；
- 地址查询：`getsockname`、`getpeername`；
- 数据收发：`sendto`、`recvfrom`、`sendmsg`、`recvmsg`、`sendmmsg`、`recvmmsg`、`recvmmsg_time64`；
- 选项与控制：`setsockopt`、`getsockopt`、`socket` 相关 `ioctl`。

9.4.2 `socket_file` 的状态机

`socket_file` 用 `SocketState` 描述生命周期：`CREATED`、`BOUND`、`CONNECTING`、`LISTENING`、`CONNECTED`、`CLOSED`。不同协议使用同一套外层状态，但内部

队列不同：TCP 使用 `_pending_connections` 和 `_recv_buffer`，UDP 使用 `_datagram_queue`，AF_UNIX 使用路径表或 `socketpair()` 直接建立 peer。

```
enum class SocketState
{
    CREATED,
    BOUND,
    CONNECTING,
    LISTENING,
    CONNECTED,
    CLOSED
};
```

`read()` 和 `write()` 在 `socket` 上分别映射到 `recv()` 和 `send()`。这保证了大量只使用文件接口的用户态代码也能操作 `socket`，例如 `shell` 管道重定向、库函数封装或某些测试程序中的通用 `fd` 读写逻辑。

就绪通知同样放在 `socket_file` 中实现。TCP 读就绪取决于接收缓冲区是否有数据、对端是否关闭写半边或本端是否已进入 EOF；UDP 读就绪取决于 `datagram` 队列是否非空；`listener` 读就绪取决于 `pending` 连接队列是否有成员。写就绪则要考虑 `socket` 是否连接、写半边是否关闭、`peer` 是否存在，以及 TCP `loopback` 对端接收缓冲是否低于 512KB 上限。

这种设计与第七章的 `epoll` 实现天然衔接：`epoll_file` 不需要理解网络协议，只要回调被关注 `fd` 的 `read_ready()` 和 `write_ready()` 即可。

9.4.3 批量消息与选项兼容

为了适配真实用户态网络程序，F7LY 补齐了 `sendmsg`、`recvmsg`、`sendmmsg` 和 `recvmmsg`。`sendmsg` / `recvmsg` 负责处理 `iovec` 数组和可选地址；批量接口则循环调用单条消息路径。`recvmmsg` 有一个重要细节：第一条消息可以阻塞等待，但读到第一条后会给后续消息自动加上 `MSG_DONTWAIT`，避免用户给出较大的 `vlen` 时在队列被读空后永久卡住。

`socket option` 方面，`loopback` 后端对常见选项采取“能产生真实行为的真正实现，调优类选项稳定 no-op”的策略。`SO_RCVTIMEO` 会影响阻塞接收等待，`SO_SNDTIMEO` 会被保存并可由用户态查询，`SO_REUSEADDR` 会影响绑定行为；`TCP_NODELAY`、`TCP_CORK`、部分缓冲区大小和拥塞控制查询在 `loopback` 中没有真

实网卡或拥塞窗口语义，因此主要返回兼容值，保证用户态库不会因为选项缺失而失败。

网络 `ioctl` 由 `SocketIoctlCompat` 提供兼容视图。当前它只暴露内核确实能模拟的 loopback 接口，`SIOCGIFCONF` 返回 `lo` 和 `127.0.0.1`，`SIOCGIFFLAGS` 返回 `IFF_UP | IFF_LOOPBACK | IFF_RUNNING`，`SIOCSIFFLAGS` 对 `lo` 接受为成功，对未知接口返回 `ENODEV`。这种保守实现可以满足常见探测逻辑，同时避免声称存在尚未完整实现的外部网卡控制面。

```
static constexpr short k_loopback_flags =
    k_iff_up | k_iff_loopback | k_iff_running;

void SocketIoctlCompat::fill_loopback_ifreq(absl::SocketIfreq &req)
{
    memset(&req, 0, sizeof(req));
    req.ifr_name[0] = 'l';
    req.ifr_name[1] = 'o';

    sockaddr_in addr{};
    addr.sin_family = AF_INET;
    addr.sin_addr = 0x01000007f;
    memcpy(&req.ifr_addr, &addr, sizeof(addr));
}
```

9.4.4 AF_UNIX 与 socketpair

除了 IPv4/IPv6 loopback，`socket_file` 还实现了 `AF_UNIX` 本地通信。pathname socket 在 `bind()` 时会通过 VFS 创建 `S_IFSOCK` 节点，并登记到 Unix 路径表；abstract socket 不落盘，只使用内核内 key。`socketpair()` 则绕过路径表，直接创建两个 `socket_file` 并互设 peer。

`AF_UNIX` stream 和 seqpacket 当前复用可靠本地 stream 队列，与 TCP loopback 使用相似的 `_peer`、`_recv_buffer`、阻塞等待和半关闭机制。这样做减少了本地可靠字节流的重复实现，也让 `socketpair()` 可以自然参与 `poll`、`epoll` 和非阻塞 I/O。

9.5 小结

F7LY 网络模块当前形成了清晰的“双路径”结构：loopback TCP/UDP/AF_UNIX 是已经落地并用于评测的内核内数据面，负责 localhost 场景下的真实 payload、阻塞/非阻塞、超时、信号中断和就绪通知；ONPS + VirtIO-Net 是外部 IPv4 网络后端，负责把协议栈与真实 VirtIO 网卡连接起来。

这种结构使网络模块在工程上保持了可验证性和可扩展性。可验证性来自 loopback：它绕开外部网络环境差异，直接服务于 iperf、netperf、LTP 和用户态库的核心 socket 语义。可扩展性来自 ONPS/VirtIO 适配层：设备驱动只处理帧和 virtqueue，协议栈只处理网络协议，Socket 层只处理 Linux ABI 和 fd 语义。三者边界明确，后续无论是继续增强外网 TCP/UDP，还是补充更多 ioctl、IPv6 和网卡配置能力，都可以在现有分层上推进。

第十章 总结和展望

2026 年的 F7LY-OS 已经从“能够启动并跑通基础测例”的教学内核，推进到面向 Linux ABI、双架构连续测例和复杂用户程序的系统原型。内核主体仍保持 C++23 freestanding 的实现方式，但系统边界已经明显扩大：RISC-V 与 LoongArch 共享同一套内核服务，用户态同时覆盖 musl 与 glibc 动态链接程序，根文件系统以 ext4 为主，辅以 FAT32 数据盘、虚拟文件系统、socket、epoll、eventfd、memfd、futex、信号、POSIX timer 这些现代 Linux 程序常用接口。

10.1 已验证的功能

10.1.1 连续测例通过情况

截至当前 scoreboard 小分文件记录，F7LY-OS 在四组合评测口径下共登记 12892 条组合测例。这里的“组合测例”按架构和 C 运行库分别计数，即同一个测例在 RISC-V + musl、RISC-V + glibc、LoongArch + musl、LoongArch + glibc 下会被视为不同验证项。

当前连续测例覆盖以下类型：

- **basic**: 四组合共 128 条组合测例，覆盖基础进程、文件、目录、时间、内存映射、管道、挂载与卸载接口。
- **busybox**: 四组合共 220 条组合测例，覆盖 BusyBox applet、shell 脚本、常用命令、文件浏览和命令行环境初始化。
- **cyclicttest**: 四组合共 20 条组合测例，覆盖实时性和定时器延迟观测；当前 scoreboard 尚未记录通过状态。
- **iozone**: 四组合共 32 条组合测例，覆盖顺序读写、随机读、反向读、跨步读、fwrite/fread、pwrite/pread、pwritev/preadv。
- **iperf**: 四组合共 32 条组合测例，覆盖 TCP/UDP 吞吐、socket 创建、连接、读写、关闭和脚本化网络测试；当前 scoreboard 尚未记录通过状态。
- **libcbench**: 四组合共 4 条组合测例，覆盖 C 运行库性能路径、线程库、内存分配和常用 libc 接口。
- **libctest-static**: 两组合共 428 条组合测例，覆盖静态链接入口下的 libc 行为、pthread、TLS、数学函数、stdio、文件接口和进程接口。

- **libctest-dynamic**: 两组合共 440 条组合测例，覆盖动态链接入口下的 libc 行为、动态装载器、共享库解析、pthread、TLS、stdio、文件接口和进程接口。
- **lmbench**: 四组合共 96 条组合测例，覆盖系统调用延迟、上下文切换、管道、文件、内存、进程创建和通信微基准；
- **LTP**: 四组合共 11320 条组合测例，是当前最大规模的 Linux ABI 连续测例集合；scoreboard 已记录 853 条单测例通过记录。
- **lua**: 四组合共 36 条组合测例，覆盖解释器启动、脚本执行、文件 I/O、随机数、字符串、排序和数学函数。
- **netperf**: 四组合共 24 条组合测例，覆盖 TCP_STREAM、TCP_RR、UDP_STREAM、UDP_RR、socket option 和网络吞吐/延迟脚本；

LTP 已确认通过项覆盖 fork/exec/wait、mmap/munmap/mprotect、open/read/write/stat、signal、futex、timer、socket、epoll 这些核心 Linux ABI 行为。其他测例类型的 scoreboard 记录正在继续维护。

这些结果说明当前内核已经具备运行真实用户态连续测例套件的基础稳定性：测例不再只验证单个 syscall 的存在，而是在动态链接器、文件系统、地址空间、线程同步、信号恢复和进程生命周期之间进行组合压力测试。

10.1.2 用户程序运行能力

除自动连续测例外，F7LY-OS 已经能够支撑 5 类代表性复杂用户程序的关键运行路径：shell、gcc、rustc、vim 和 git。这些程序覆盖的不是单一接口，而是现代 Unix 用户态环境的典型组合需求：命令解析与进程派生、阶段化编译、终端控制、多线程同步、内存映射、文件系统元数据、仓库对象管理以及 socket 网络通信。

表 10-1 代表性用户程序及其覆盖的关键内核能力

程序	特征	关键依赖与说明
shell	程序启动	解析命令，通过 <code>execve</code> / <code>execveat</code> 启动外部程序。
	重定向与管道	依赖 <code>pipe2</code> 、 <code>dup</code> / <code>dup3</code> 、 <code>openat</code> 、 <code>close</code> 实现 <code> </code> 、 <code>></code> 、 <code><</code> 这些语法。
	作业控制	依赖 <code>setpgid</code> 、 <code>setsid</code> 、 <code>ioctl</code> (<code>TIOCSPGRP</code> / <code>TIOCGPGRP</code>) 支持前台/后台任务、 <code>fg</code> 、 <code>bg</code> 、 <code>Ctrl-Z</code> 。
	脚本解释器	依赖内核在 <code>execve</code> 中解析 <code>shebang</code> 。
gcc	阶段调度	<code>gcc</code> 本身更像 <code>driver</code> ，真正工作交给 <code>cc1</code> 、 <code>as</code> 、 <code>ld</code> 这些子程序。
	阶段通信	依赖 <code>pipe2</code> 、 <code>dup</code> / <code>dup3</code> 把预处理、编译、汇编、链接阶段连接起来。
	文件读写	依赖 <code>openat</code> 、 <code>read</code> 、 <code>write</code> 、 <code>lseek</code> 、 <code>pread64</code> / <code>pwrite64</code> 处理源码、中间文件、目标文件。
	临时产物	依赖 <code>fstat</code> / <code>fstatat</code> 、 <code>statx</code> 、 <code>unlinkat</code> 、 <code>renameat2</code> 管理临时产物和输出文件。
rustc	内存映射	频繁使用 <code>mmap</code> 、 <code>munmap</code> 、 <code>mremap</code> 处理 LLVM 后端和大型编译数据结构。
	权限切换	依赖 <code>mprotect</code> 切换映射区域权限，例如 <code>PROT_READ</code> / <code>PROT_EXEC</code> 。
	并行编译	依赖 <code>futex</code> 、 <code>set tid address</code> 、 <code>set robust list</code> 支撑多线程同步。
	产物缓存	可能使用 <code>ftruncate</code> 、 <code>fallocate</code> 、 <code>sendfile</code> 、 <code>getrandom</code> 处理缓存、产物和哈希相关逻辑。
vim	raw 模式	核心特征是 <code>ioctl</code> (<code>TCGETS</code> / <code>TCSETS</code> / <code>TCSETSW</code>)，进入编辑模式前保存终端设置，退出时恢复。
	窗口尺寸	依赖 <code>ioctl</code> (<code>TIOCGWINSZ</code>) 获取行列数，支撑全屏编辑界面。
	控制终端	依赖 <code>ioctl</code> (<code>TIOCGPGRP</code> / <code>TIOCSPGRP</code> / <code>TIOCSTTY</code> / <code>TIOCGSID</code>) 正确接管终端。
	事件监听	依赖 <code>ppoll</code> 、 <code>pselect6</code> 阻塞接收键盘输入、信号和终端事件。
	文件状态	依赖 <code>getdents64</code> 、 <code>fstat</code> 支持文件浏览、打开、保存这些编辑器功能。
git	工作区操作	大量使用 <code>getdents64</code> 、 <code>mkdirat</code> 、 <code>unlinkat</code> 、 <code>linkat</code> 、 <code>symlinkat</code> 、 <code>renameat2</code> 管理工作区和 <code>.git</code> 目录。
	pack 读取	依赖 <code>mmap</code> 、 <code>madvise</code> 快速读取和解析大型 <code>pack/index</code> 文件。
	索引锁	依赖 <code>flock</code> 、 <code>fcntl</code> 管理 <code>.git/index.lock</code> 这类锁文件，避免并发破坏仓库状态。
	网络传输	<code>clone</code> / <code>pull</code> / <code>push</code> 依赖 <code>socket</code> 、 <code>connect</code> 、 <code>sendmsg</code> / <code>recvmsg</code> 、 <code>setsockopt</code> / <code>getsockopt</code> 。
	路径权限	依赖 <code>getcwd</code> 、 <code>chdir</code> 、 <code>umask</code> 、 <code>fchmodat</code> 处理仓库根目录、配置和文件权限。

这 5 类程序共同构成了比单个 benchmark 更严格的验证面。shell 证明内核可以作为用户程序启动中枢；gcc 和 rustc 证明动态链接、文件系统、管道和地址空间管理能够承载大型工具链；vim 证明终端、信号和交互式事件监听具备可用基础；git 则把 VFS、mmap、锁、路径权限和网络连接组合在一起，检验内核对真实开发工作流的支持程度。

10.2 AI 工具使用说明

本项目在 2026 年开发过程中使用了 AI Agent 辅助工程开发。主要工具为 VSCode / Codex 交互环境中的 Codex，相关大模型为 GPT-5。AI 工具的定位是工程协作者：用于阅读代码、定位模块、生成候选补丁、分析日志、整理连续测例结果、维护项目文档草稿。系统设计、需求取舍、最终代码合入、测例是否计入通过、提交记录与答辩材料由参赛队员负责。

为避免 AI 工具零散介入导致工程过程不可追踪，项目建立了一套固定协作流程：

表 10-2 AI Agent 协作流程

阶段	机制	说明
1	规则入口	所有 AI Agent 在行动前先读取 <code>AGENTS.md</code> ，遵循沟通语言、Git 操作边界、Python 环境、日志保存、scoreboard 更新、危险操作禁令和人工验收规则。
2	任务路由	根据任务类型读取 <code>agent_docs/</code> 下的配套文档：架构定位读取 <code>project_architecture.md</code> ，构建调试读取 <code>development_debugging.md</code> ，连续测例状态维护读取 <code>scoreboard.md</code> 与 <code>scoreboard/README.md</code> 。
3	计划记录	功能开发、性能优化、平台适配和测例修复先在 <code>plan_docs/</code> 中建立或更新任务计划，写清目标、阶段、验证点和风险；完成后只添加“已完成，待验收”标记。
4	实现与验证	AI Agent 可以生成候选代码补丁、Typst 文档草稿和调试命令，但每次修改后必须执行语法检查、构建或窄验证；QEMU 长输出写入 <code>logs/</code> 或 <code>logs/run/</code> 。
5	人工验收	参赛队员检查 diff、验证日志和实际运行效果后决定是否接受修改；未经人工确认，AI Agent 不提交 commit、不推送远端、不改写历史。
6	成果入账	若某个连续测例被验证通过，先在 scoreboard 对应架构、libc 和小分文件中写入 <code>PASS</code> 及简短依据，再由生成器刷新汇总。

这套流程把 AI 工具的贡献分成三类可追踪成果：第一类是任务计划与交互摘要，保存在 `plan_docs/`；第二类是候选实现和文档修改，体现在 `git diff` 和后续 `commit` 记录中；第三类是测例验证结果，沉淀在 `logs/`、`logs/run/` 和 `scoreboard/`。因此，AI 工具参与的工作不会绕过人工 review，也不会直接替代参赛队员对正确性、可维护性和比赛诚信披露的责任。

10.3 总结与后续工作

整体来看，F7LY-OS 2026 年的核心成果是把若干原本分散的内核能力收束成一套面向 Linux 用户态程序的运行环境。进程、线程、地址空间、文件系统、网络 and 系统调用不再是彼此孤立的模块，而是围绕真实程序的启动、运行、同步、I/O 和退出路径协同工作。尤其是动态 ELF 装载、musl/glibc loader 路径重写、shebang、VMA/VmObject、futex、epoll、socket_file 和 VFS file 多态体系，使内核可以用更接近 Linux 的方式承接复杂应用。

后续工作仍应围绕“减少特殊分支、提高长连续测例稳定性、扩大真实应用覆盖面”展开。首先，LTP 中尚未通过或暂未确认的项目需要继续按 syscall 族和子系统聚类推进，避免只修单点而破坏其他组合。其次，glibc 侧 libctest 和大型工具链应用对 ABI 细节更敏感，仍需要继续校准 statx、termios、socket option、资源限制、命名空间视图和 `/proc / /sys` 虚拟文件内容。最后，git 网络、vim 全屏交互、gcc/rustc 完整编译链这些真实工作流应逐步从“能启动、能覆盖关键路径”推进到“能稳定完成端到端任务”，让 F7LY-OS 的评测能力进一步转化为真实用户态生态能力。

第十一章 附录

11.1 附录 A PCB 字段总表

`proc::Pcb` 是 F7LY 用来承载进程与线程全部内核态元数据的统一对象。系统中的所有任务都以 PCB 形式存在于全局进程池中，由调度、信号、文件系统和地址空间等子系统共同引用。

表 11-1 PCB 字段总表

字段名	含义
标识与状态	
<code>_lock</code>	PCB 自旋锁，保护并发访问
<code>_parent</code>	父进程 PCB 指针，与 <code>_ppid</code> 互补
<code>_pid</code> , <code>_tid</code> , <code>_tgid</code> , <code>_ppid</code> , <code>_pgid</code>	进程号、线程号、线程组号、父进程号、进程组号
<code>_global_id</code>	进程池槽位索引， <code>alloc_proc()</code> 分配时确定
<code>_sid</code>	会话 ID， <code>setsid()</code> 设置
<code>_uid</code> / <code>_euid</code> / <code>_suid</code> / <code>_fsuid</code>	四组用户 ID (real / effective / saved / filesystem)
<code>_gid</code> / <code>_egid</code> / <code>_sgid</code> / <code>_fsgid</code>	四组组 ID
<code>_cap_*[2]</code>	Linux capability 位图 (effective / permitted / inheritable / ambient / bounding)，覆盖 41 种能力
进程状态	
<code>_state</code>	进 程 状 态 : UNUSED → USED → RUNNABLE ↔ RUNNING → SLEEPING / ZOMBIE / STOPPED
<code>_chan</code>	睡眠等待通道， <code>sleep()</code> 设置、 <code>wakeup()</code> 匹配
<code>_killed</code> , <code>_exiting</code>	终止标志；退出清理中标记（禁止 timer 抢占）
<code>_xstate</code>	退出码，父进程通过 <code>wait4</code> / <code>waitid</code> 获取
<code>_parent_exit_signal</code>	退出时发往父进程的信号，默认 SIGCHLD，线程创建时为 0
地址空间与文件	

字段名	含义
<code>_memory_manager</code>	<code>ProcessMemoryManager*</code> ，封装页表、VMA、堆、mmap 区域。 <code>CLONE_VM</code> 时共享（引用计数）， <code>fork</code> 时深拷贝
<code>_kstack</code>	内核栈虚拟地址，由 <code>_global_id</code> 换算，通常 2 页 + 1 页 guard page
<code>_trapframe</code>	<code>TrapFrame*</code> ，用户态寄存器保存区（一页物理内存），陷入内核时保存 GPR 等现场
<code>_context</code>	<code>Context</code> ，内核态上下文（ <code>ra / sp / callee-saved</code> 寄存器）， <code>swtch()</code> 在此保存/恢复调度现场
<code>_ofile</code>	文件描述符表（ <code>ofile*</code> ），容量 1024。 <code>CLONE_FILES</code> 时共享并增加引用计数
<code>_cwd, _root_name</code>	当前工作目录； <code>chroot</code> 根目录
<code>_umask</code>	文件创建权限掩码
<code>_personality</code>	Linux personality(2) 状态
信号与同步	
<code>_sigactions</code>	<code>sigaction_struct*</code> ， <code>sigaction</code> 表， <code>CLONE_SIGHAND</code> 时共享。 <code>exec</code> 时非 <code>SIG_IGN</code> 的 handler 重置为默认
<code>_sigmask</code>	<code>uint64</code> ，信号屏蔽掩码（64-bit 位图）， <code>sigprocmask</code> 修改
<code>_signal</code>	<code>uint64</code> ，待处理信号掩码， <code>add_signal()</code> 设置， <code>clear_signal()</code> 清除
<code>_siginfo_mask, _queue_siginfo</code>	掩码 <code>siginfo_t</code> 排队信息，每个 <code>signal</code> 保留最后一份
<code>sig_frame</code>	<code>signal_frame*</code> ，信号栈帧链表头。 <code>do_handle()</code> 压入， <code>sig_return()</code> 弹出并恢复 <code>TrapFrame</code>
<code>_alt_stack, _on_sigstack</code>	备用信号栈（ <code>sigaltstack</code> 设置）及当前是否在其上执行
<code>_futex_addr</code>	<code>futex</code> 等待的用户态地址（调试用）
<code>_futex_key</code>	<code>futex</code> 匹配键，按用户态地址映射到的物理地址计算，跨进程共享映射可正确匹配
<code>_vfork_parent</code>	<code>CLONE_VFORK</code> 子进程释放地址空间前需要唤醒的父进程
线程相关	
<code>_clear_tid_addr</code>	<code>CLONE_CHILD_CLEAR_TID</code> 指定的用户态地址。线程退出时清零 4 字节并 <code>futex_wake</code> ，供 <code>pthread_join</code> 使用

字段名	含义
_robust_list, _robust_mutex	健壮 mutex 链表头及其用户态地址。线程异常退出时遍历链表，对 owned mutex 标记 FUTEX_OWNER_DIED 并唤醒等待者
调度与 I/O	
_priority	nice 值 [-20, 19]，数值越小优先级越高。同时作用于 CPU 调度和块层 I/O 排队
_sched_policy, _sched_priority	Linux sched ABI 策略与实时优先级；_sched_reset_on_fork 控制 fork 后是否恢复 SCHED_OTHER
_io_priority_override, _has_io_priority_override	块层 I/O 优先级覆盖值及其启用标记
_cpu_mask	CPU 亲和性位图，每个 bit 对应一个可用 CPU
_slot	当前时间片剩余量
_rlim_vec[16]	16 类 rlimit (CPU / FSIZE / NOFILE / NPROC 等)，prlimit64 系统调用读写

11.2 附录 B 系统调用实现列表

11.2.1 B.1 进程管理

表 11-2 B.1 进程管理类系统调用

函数	说明
进程创建与执行	
sys_fork()	深拷贝地址空间，创建子进程
sys_clone()	按 clone flags 创建线程或进程
sys_clone3()	clone 扩展版本，支持更多 flags
sys_execve()	解析 ELF 或 shebang，装载并执行
sys_execveat()	execve 扩展版本，支持目录 fd 相对路径
sys_exec()	旧版执行接口，兼容早期二进制
进程退出与等待	
sys_exit()	当前线程退出
sys_exit_group()	线程组全部退出
sys_wait4()	等待子进程状态变化
sys_waitid()	等待子进程，支持更细粒度选项

函数	说明
sys_wait()	旧版等待接口，兼容早期二进制
进程标识	
sys_getpid()	获取进程号
sys_getppid()	获取父进程号
sys_gettid()	获取线程号
sys_personality()	设置进程执行域
用户与组凭据	
sys_getuid()	获取实际用户 ID
sys_geteuid()	获取有效用户 ID
sys_getgid()	获取实际组 ID
sys_getegid()	获取有效组 ID
sys_setuid()	设置用户 ID
sys_setgid()	设置组 ID
sys_setreuid()	设置实际和有效用户 ID
sys_setregid()	设置实际和有效组 ID
sys_setresuid()	设置实际、有效和保存用户 ID
sys_setresgid()	设置实际、有效和保存组 ID
sys_getresuid()	获取实际、有效和保存用户 ID
sys_getresgid()	获取实际、有效和保存组 ID
sys_setfsuid()	设置文件系统用户 ID
sys_setfsgid()	设置文件系统组 ID
sys_getgroups()	获取附加组列表
sys_setgroups()	设置附加组列表
会话与进程组	
sys_getpgid()	获取进程组号
sys_setpgid()	设置进程组号
sys_setsid()	创建新会话
sys_getsid()	获取会话 ID
调度与优先级	
sys_getpriority()	获取 nice 值
sys_setpriority()	设置 nice 值
sys_sched_yield()	主动让出 CPU
sys_sched_getaffinity()	获取 CPU 亲和性
sys_sched_setaffinity()	设置 CPU 亲和性

函数	说明
sys_sched_getparam()	获取调度参数
sys_sched_setparam()	设置调度参数
sys_sched_getscheduler()	获取调度策略
sys_sched_setscheduler()	设置调度策略
sys_getrusage()	获取资源使用统计
线程支持	
sys_set_tid_address()	设置 clear_child_tid 地址
sys_set_robust_list()	设置 futex robust list 头
sys_get_robust_list()	获取 futex robust list 头
sys_unshare()	取消共享命名空间等资源
资源限制与控制	
sys_prlimit64()	获取或设置资源限制
sys_prctl()	进程控制操作
sys_umask()	设置文件创建权限掩码

11.2.2 B.2 内存管理

表 11-3 B.2 内存管理类系统调用

函数	说明
堆管理	
sys_brk()	调整 heap 末端
内存映射	
sys_mmap()	创建文件映射或匿名映射
sys_munmap()	解除映射
sys_mremap()	调整已有映射的大小或位置
sys_mprotect()	修改映射页权限
sys_madvise()	通知内核内存使用意图
sys_msync()	同步 mmap 脏页到文件
sys_mlock()	锁定内存页禁止换出
sys_munlock()	解除内存页锁定
sys_readahead()	预读文件数据到页缓存
sys_get_mempolicy()	获取 NUMA 内存策略
共享内存	
sys_shmget()	创建或查找共享内存段

函数	说明
sys_shmat()	附加共享内存段到地址空间
sys_shmdt()	解除共享内存段附加
sys_shmctl()	控制共享内存段
匿名内存	
sys_memfd_create()	创建匿名内存文件

11.2.3 B.3 文件系统

表 11-4 B.3 文件系统类系统调用

函数	说明
文件打开与关闭	
sys_openat()	打开或创建文件
sys_openat2()	openat 扩展版本
sys_close()	关闭文件描述符
sys_close_range()	批量关闭文件描述符
读写	
sys_read()	读文件
sys_write()	写文件
sys_readv()	矢量读
sys_writev()	矢量写
sys_pread64()	指定偏移读
sys_pwrite64()	指定偏移写
sys_preadv()	指定偏移矢量读
sys_pwritev()	指定偏移矢量写
sys_lseek()	移动文件偏移
文件元数据	
sys_fstat()	通过 fd 获取文件元数据
sys_fstatat()	通过目录 fd + 路径获取元数据
sys_statx()	扩展元数据查询
sys_truncate()	按路径截断文件
sys_ftruncate()	按 fd 截断文件
sys_utimensat()	设置文件时间戳
sys_fallocate()	预分配文件空间
目录操作	

函数	说明
sys_mkdirat()	创建目录
sys_getdents64()	读取目录条目
sys_getcwd()	获取工作目录
sys_chdir()	切换工作目录
sys_fchdir()	通过 fd 切换工作目录
链接与重命名	
sys_linkat()	创建硬链接
sys_unlinkat()	删除硬链接
sys_symlinkat()	创建符号链接
sys_readlinkat()	通过目录 fd 读取符号链接
sys_renameat2()	重命名文件或目录
挂载	
sys_mount()	挂载文件系统
sys_umount2()	卸载文件系统
sys_statfs()	获取文件系统统计
sys_fstatfs()	通过 fd 获取文件系统统计
文件控制	
sys_fcntl()	文件锁、CLOEXEC、管道容量等
sys_ioctl()	设备控制
sys_dup()	复制 fd
sys_dup3()	复制 fd 到指定编号
sys_flock()	文件锁
同步	
sys_sync()	全部缓存同步到磁盘
sys_fsync()	单文件缓存同步
sys_fdatasync()	单文件数据同步
sys_sync_file_range()	指定范围同步到磁盘
高级 I/O	
sys_splice()	两 fd 间搬运数据
sys_sendfile()	文件到 socket 直接发送
sys_copy_file_range()	文件间直接复制数据
文件权限	
sys_fchmod()	修改文件权限
sys_fchmodat()	按路径修改文件权限

函数	说明
sys_fchown()	修改文件所有者
sys_fchownat()	按路径修改文件所有者
sys_faccessat()	检查文件访问权限
sys_faccessat2()	faccessat 扩展版本
sys_fchmodat2()	fchmodat 扩展版本
其他	
sys_pipe2()	创建管道
sys_mknodat()	创建设备文件或 FIFO
sys_mknod()	旧版设备文件创建接口

11.2.4 B.4 信号

表 11-5 B.4 信号类系统调用

函数	说明
sys_kill()	向进程发送信号
sys_tkill()	向线程发送信号
sys_tgkill()	向线程组指定线程发送信号
sys_rt_sigaction()	注册或查询信号处理函数
sys_rt_sigprocmask()	阻塞或解除阻塞信号
sys_rt_sigpending()	查询阻塞的待处理信号
sys_rt_sigsuspend()	原子替换 mask 并等待信号
sys_rt_sigreturn()	从信号处理函数返回
sys_rt_sigtimedwait()	带超时的等待信号
sys_sigaltstack()	设置备用信号栈
sys_rt_sigqueueinfo()	向进程发送带数据的信号
sys_kill_signal()	实时信号发送接口
sys_signalfd4()	创建信号接收 fd

11.2.5 B.5 时间与定时器

表 11-6 B.5 时间与定时器类系统调用

函数	说明
时间获取与设置	
sys_clock_gettime()	获取指定时钟的时间

函数	说明
sys_clock_settime()	设置指定时钟的时间
sys_clock_getres()	获取指定时钟的分辨率
sys_gettimeofday()	获取墙上时间
sys_times()	返回进程 CPU 时间
睡眠	
sys_nanosleep()	睡眠指定纳秒
sys_clock_nanosleep()	基于指定时钟的高精度睡眠
POSIX 定时器	
sys_timer_create()	创建 POSIX 定时器
sys_timer_settime()	设置定时器到期时间
sys_timer_gettime()	获取定时器剩余时间
sys_timer_delete()	删除定时器
sys_setitimer()	设置间隔定时器
sys_getitimer()	获取间隔定时器状态
时间校准	
sys_adjtimex()	调整内核时钟参数
sys_clockadjtime()	调整指定时钟参数
fd 定时器	
sys_timerfd_create()	创建 fd 定时器

11.2.6 B.6 Futex

表 11-7 B.6 Futex 类系统调用

函数	说明
sys_futex()	快速用户空间互斥锁：WAIT / WAKE / REQUEUE 等子操作

11.2.7 B.7 事件通知

表 11-8 B.7 事件通知类系统调用

函数	说明
sys_epoll_create1()	创建 epoll 实例
sys_epoll_ctl()	添加 / 修改 / 删除关注 fd
sys_epoll_pwait()	等待关注 fd 就绪

函数	说明
sys_epoll_pwait2()	epoll_pwait 扩展版本
sys_ppoll()	轮询 fd 就绪状态（支持信号掩码）
sys_pselect6()	通知 fd 就绪状态（支持信号掩码）
sys_eventfd2()	创建事件通知 fd

11.2.8 B.8 网络

表 11-9 B.8 网络类系统调用

函数	说明
sys_socket()	创建 socket
sys_socketpair()	创建一对已连接 socket
sys_bind()	绑定本地地址
sys_listen()	标记监听状态
sys_accept()	取出一个连接
sys_accept4()	accept 带标志位
sys_connect()	发起连接
sys_sendto()	UDP 发送
sys_recvfrom()	UDP 接收
sys_sendmsg()	矢量消息发送
sys_recvmsg()	矢量消息接收
sys_sendmmsg()	批量发送消息
sys_recvmmsg()	批量接收消息
sys_recvmmsg_time64()	批量接收消息（time64 变体）
sys_getsockname()	获取本地地址
sys_getpeername()	获取对端地址
sys_setsockopt()	设置 socket 选项
sys_getsockopt()	获取 socket 选项
sys_shutdown_socket()	关闭读端 / 写端

11.2.9 B.9 权限

表 11-10 B.9 权限类系统调用

函数	说明
sys_capget()	获取线程能力集

函数	说明
sys_capset()	设置线程能力集

11.2.10 B.10 系统信息与其他

表 11-11 B.10 系统信息与其他系统调用

函数	说明
sys_uname()	返回内核信息
sys_sysinfo()	返回系统统计
sys_syslog()	内核日志操作
sys_getrandom()	获取随机数
sys_reboot()	重启或关机
sys_shutdown()	系统关机（内部接口）
sys_sethostname()	设置主机名
sys_setdomainname()	设置域名
sys_strerror()	获取错误码描述字符串
sys_perror()	打印错误信息
sys_ptrace()	进程跟踪与调试
sys_setns()	加入命名空间
sys_acct()	进程记账开关
sys_chroot()	切换根目录
调试与内部接口	
sys_sleep()	内核级睡眠（调试用）
sys_uptime()	获取内核运行时间（调试用）
sys_userdebug1()	用户调试接口 1
sys_userdebug2()	用户调试接口 2
sys_userdebug3()	用户调试接口 3
sys_userdebug4()	用户调试接口 4
sys_userdebug5()	用户调试接口 5

11.2.11 B.11 扩展属性 (xattr)

表 11-12 B.11 扩展属性类系统调用

函数	说明
sys_setxattr()	设置扩展属性

函数	说明
sys_lsetxattr()	设置扩展属性（不跟随符号链接）
sys_fsetxattr()	通过 fd 设置扩展属性
sys_getxattr()	获取扩展属性
sys_lgetxattr()	获取扩展属性（不跟随符号链接）
sys_fgetxattr()	通过 fd 获取扩展属性
sys_listxattr()	列出扩展属性
sys_llistxattr()	列出扩展属性（不跟随符号链接）
sys_flistxattr()	通过 fd 列出扩展属性
sys_removexattr()	删除扩展属性
sys_lremovexattr()	删除扩展属性（不跟随符号链接）
sys_fremovexattr()	通过 fd 删除扩展属性

11.2.12 B.12 文件监控

表 11-13 B.12 文件监控类系统调用

函数	说明
sys_inotify_init1()	创建 inotify 实例
sys_inotify_add_watch()	添加监控目标
sys_inotify_rm_watch()	删除监控目标
sys_fanotify_init()	创建 fanotify 实例
sys_fanotify_mark()	添加或删除 fanotify 监控标记

11.2.13 B.13 SysV 信号量

表 11-14 B.13 SysV 信号量类系统调用

函数	说明
sys_semget()	创建或查找信号量集
sys_semctl()	控制信号量集
sys_semop()	信号量操作
sys_semtimedop()	带超时的信号量操作

11.2.14 B.14 其他高级接口

表 11-15 B.14 其他高级接口

函数	说明
sys_bpf()	BPF 程序加载与操作
sys_io_uring_setup()	创建 io_uring 实例
sys_perf_event_open()	打开性能事件
sys_userfaultfd()	创建用户态缺页处理 fd
sys_membarrier()	内存屏障同步
sys_memfd_secret()	创建机密内存文件
sys_name_to_handle_at()	文件路径转句柄
sys_open_tree()	打开挂载树文件描述符
sys_fsopen()	打开文件系统配置上下文
sys_fsconfig()	配置文件系统上下文
sys_fspick()	选取已存在的文件系统配置
sys_pidfd_open()	通过 pid 获取 fd
sys_pidfd_send_signal()	通过 pidfd 发送信号
sys_keyctl()	内核密钥管理
sys_add_key()	添加内核密钥
sys_vmsplice()	内存到管道的矢量搬运
sys_fadvise64()	通知内核文件访问模式建议
sys_remap_file_pages()	重映射文件页（已废弃）

11.3 附录 C 虚拟文件与特殊设备节点

F7LY 的虚拟文件由 VFS 初始化阶段注册到内存目录树中。多数节点在 `open` 时创建 `virtual_file`，读取时由对应 `provider` 动态生成内容；少数 `/dev` 节点表现为设备文件，读写或 `ioctl` 会转入对应设备逻辑。正文只介绍目录职责，本附录列出当前内核暴露的代表性节点。

表 11-16 C.1 procfs 与 sysctl 虚拟节点

目录	代表节点	职责
<code>/proc/self</code>	<code>exe</code> 、 <code>cmdline</code> 、 <code>stat</code> 、 <code>status</code> 、 <code>maps</code> 、 <code>pagemap</code>	暴露当前进程的可执行文件、命令行、状态统计、地址空间布局和页映射信息。
<code>/proc/self/ns</code>	<code>time</code> 、 <code>time_for_children</code> 、 <code>timens_offsets</code>	提供时间命名空间句柄和偏移视图，使用户态可以按

目录	代表节点	职责
		Linux ABI 查询或调整相关状态。
/proc/ <pid>	<pid>/stat 、 <pid>/fdinfo/<fd> 、 <pid>/ns/mnt	按路径动态识别目标进程，生成进程状态、fd 信息和 mount namespace 句柄。
/proc	meminfo 、 cpuinfo 、 version 、 uptime 、 stat 、 interrupts 、 mounts 、 self/mounts	提供系统内存、CPU、版本、运行时间、中断和挂载表等运行状态。
/proc/sys/ kernel	pid_max 、 shmmax 、 shmmni 、 shmall 、 shm_next_id 、 tainted 、 domainname 、 random/entropy_avail	暴露进程号、System V 共享内存、内核状态、域名和随机熵等可查询或可写参数。
/proc/sys/ fs	pipe-user-pages-soft 、 pipe-max-size 、 lease-break-time 、 inotify/max_queued_events 、 inotify/ max_user_instances	提供管道容量、文件 lease 和 inotify 限制等文件系统相关参数。
/proc/sys/ net	ipv4/conf/default/tag 、 ipv4/conf/lo/tag	提供 IPv4 配置视图中的标签节点。
/proc/ sysvipc	shm	导出当前 IPC namespace 可见的 System V 共享内存段表。

表 11-17 C.2 sysfs 与内置配置文件节点

目录	代表节点	职责
/sys/ devices/ system/cpu	online 、 present 、 possible 、 kernel_max 、 cpu0/online	提供单核视图下的 CPU 在线、存在和可用范围信息。
/sys/ devices/ system/ node	online 、 possible 、 has_cpu 、 has_memory 、 node0/cpulist 、 node0/cpumap 、	提供退化到 node0 的 NUMA 兼容视图，供用户态库查询拓扑。

目录	代表节点	职责
	node0/distance 、 node0/meminfo	
/etc	passwd 、 group 、 hosts 、 resolv.conf 、 protocols 、 ld.so.preload 、 ld.so.cache 、 localtime	以内核内置内容补齐基础运行环境，支撑用户、组、解析器、协议名、动态链接器和时区探测。
/boot	config-6.17.0 、 config-5.15.0-F7LY	提供内核配置视图，使用户态工具能够读取当前内核能力。

表 11-18 C.3 特殊设备节点

目录	代表节点	职责
/dev	null 、 zero 、 full 、 urandom	提供空设备、零页设备、写满错误设备和非阻塞伪随机字节设备。
/dev	loop-control 、 loop0 ~ loop63	提供 loop 设备控制面和数据面；控制节点管理实例，数据节点把块语义转发到后端文件。
/dev/block	8:0	提供块设备兼容入口，供用户态按 Linux 风格路径识别块设备。
/dev	ptmx 、 pts/0	提供最小伪终端节点，支撑终端库对 pty 路径的探测和打开。
/dev/net	tun ； 另有 /dev/tun	提供 TUN 设备兼容节点，支持网络隧道相关 ioctl 探测。
/dev	rtc 、 rtc0 、 misc/rtc	提供实时时钟兼容路径。
/dev	cpu_dma_latency	保存 CPU DMA latency 目标值，作为设备控制类 ABI 状态。